VOL. 13

AI (basics)

What artificial intelligence is, how the classic methods work, and what they cost society

Source	elementsofai.com
Generated	2026-10-10
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 13 - AI (basics)

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 8

Chapter map . . . 8

Chapter 1 - What AI is (and is not) . . . 9

1.1 Summary . . . 9

1.2 Concepts . . . 10

Two ways to build the same behaviour . . . 10

Two ways to make a decision, side by side . . . 10

The AI family tree . . . 11

What AI is not . . . 11

Narrow and general . . . 11

1.3 Examples . . . 12

1.4 Common mistakes and best practices . . . 14

1.5 Practice exercises . . . 14

Solutions . . . 15

1.6 Mini test . . . 15

Answer key and explanations . . . 16

Chapter 2 - Philosophy of AI and the Turing test . . . 16

2.1 Summary . . . 16

2.2 Concepts . . . 16

The imitation game . . . 16

What the test can and cannot settle . . . 17

The Chinese Room . . . 17

Weak and strong claims . . . 18

2.3 Examples . . . 18

2.4 Common mistakes and best practices . . . 20

2.5 Practice exercises . . . 20

Solutions . . . 21

2.6 Mini test . . . 21

Answer key and explanations . . . 22

Chapter 3 - Problem solving with search . . . 22

3.1 Summary . . . 22

3.2 Concepts . . . 23

States, actions and the state space . . . 23
Level by level, or head first? . . . 23
Why the representation matters . . . 23
3.3 Examples . . . 24
3.4 Common mistakes and best practices . . . 30
3.5 Practice exercises . . . 31
Solutions . . . 31
3.6 Mini test . . . 31
Answer key and explanations . . . 32
Chapter 4 - Game trees and minimax . . . 32
4.1 Summary . . . 32
4.2 Concepts . . . 33
The vocabulary of a game tree . . . 33
Backing values up a tree . . . 33
Minimax and alpha-beta . . . 33
When the search must stop early . . . 34
4.3 Examples . . . 34
4.4 Common mistakes and best practices . . . 39
4.5 Practice exercises . . . 40
Solutions . . . 40
4.6 Mini test . . . 41
Answer key and explanations . . . 41
Chapter 5 - Probability, odds and Bayes' rule . . . 41
5.1 Summary . . . 41
5.2 Concepts . . . 42
The basics, written out . . . 42
Bayes' rule in two clothes . . . 42
The base rate trap . . . 43
Counting instead of formulas . . . 43
5.3 Examples . . . 43
5.4 Common mistakes and best practices . . . 47
5.5 Practice exercises . . . 47
Solutions . . . 48
5.6 Mini test . . . 48
Answer key and explanations . . . 48

Chapter 6 - Naive Bayes classification in practice 49
6.1 Summary 49
6.2 Concepts 49
What the model needs 49
The formula behind the code 49
Smoothing, in numbers 50
The pipeline 50
6.3 Examples 50
6.4 Common mistakes and best practices 55
6.5 Practice exercises 56
Solutions 56
6.6 Mini test 56
Answer key and explanations 57
Chapter 7 - The types of machine learning 57
7.1 Summary 57
7.2 Concepts 58
The three main types 58
Labelled, unlabelled and rewarded 58
The words used when data is split 58
What k-means needs 58
7.3 Examples 59
7.4 Common mistakes and best practices 64
7.5 Practice exercises 64
Solutions 65
7.6 Mini test 65
Answer key and explanations 65
Chapter 8 - Nearest neighbours and regression 66
8.1 Summary 66
8.2 Concepts 66
What k-NN needs 66
Reading a distance table 66
Fitting a line 67
Scaling columns 67
8.3 Examples 67
8.4 Common mistakes and best practices 73

8.5 Practice exercises . . . 73
Solutions . . . 74
8.6 Mini test . . . 74
Answer key and explanations . . . 75
Chapter 9 - Neural network basics . . . 75
9.1 Summary . . . 75
9.2 Concepts . . . 75
One neuron . . . 76
The truth tables that matter . . . 76
Why a hidden layer helps . . . 76
Backpropagation in one line . . . 76
9.3 Examples . . . 77
9.4 Common mistakes and best practices . . . 80
9.5 Practice exercises . . . 81
Solutions . . . 81
9.6 Mini test . . . 81
Answer key and explanations . . . 82
Chapter 10 - Building and training a network . . . 82
10.1 Summary . . . 82
10.2 Concepts . . . 83
The training loop . . . 83
The ingredients, and what each one controls . . . 83
Learning rates compared on the same task . . . 83
What the hidden layer learns . . . 83
10.3 Examples . . . 84
10.4 Common mistakes and best practices . . . 90
10.5 Practice exercises . . . 90
Solutions . . . 91
10.6 Mini test . . . 91
Answer key and explanations . . . 91
Chapter 11 - Practical AI with Python . . . 92
11.1 Summary . . . 92
11.2 Concepts . . . 92
The pipeline . . . 92
Reading the confusion matrix . . . 92

Cross-validation by hand . . . 93
What to record for every run . . . 93
11.3 Examples . . . 93
11.4 Common mistakes and best practices . . . 98
11.5 Practice exercises . . . 98
Solutions . . . 99
11.6 Mini test . . . 99
Answer key and explanations . . . 99
Chapter 12 - Ethics and the society AI works in . . . 100
12.1 Summary . . . 100
12.2 Concepts . . . 100
Fairness metrics, and what each one says . . . 100
Why removing the attribute is not enough . . . 101
The feedback loop that shrinks variety . . . 101
Anonymity and quasi-identifiers . . . 101
The regulatory landscape . . . 101
12.3 Examples . . . 101
12.4 Common mistakes and best practices . . . 106
12.5 Practice exercises . . . 106
Solutions . . . 106
12.6 Mini test . . . 107
Answer key and explanations . . . 107
Chapter 13 - Where to go next . . . 108
13.1 Summary . . . 108
13.2 Concepts . . . 108
A shape for the next twelve weeks . . . 108
Choosing what to study next . . . 108
Habits that keep the skill . . . 109
13.3 Examples . . . 109
13.4 Common mistakes and best practices . . . 112
13.5 Practice exercises . . . 113
Solutions . . . 113
13.6 Mini test . . . 114
Answer key and explanations . . . 114
Cheat sheet . . . 114

14.1 Definitions . . . 114
14.2 Search . . . 115
14.3 Games . . . 115
14.4 Probability and Bayes . . . 116
14.5 Naive Bayes . . . 116
14.6 Neighbours and lines . . . 116
14.7 Neural networks . . . 117
14.8 Evaluation . . . 117
14.9 Ethics checklist . . . 118
14.10 Python quick reference . . . 118
Final test . . . 119
15.1 Final test answers . . . 120
Sources . . . 121

How to use this volume

Prerequisites. Working Python: variables, lists and dictionaries, `for` loops, `if` statements, functions, and f-strings. No mathematics beyond arithmetic, fractions and the idea of a graph on paper. Every formula is explained before it is used, and every computation is printed line by line.

Estimated study time. About 20 hours: one to two hours per chapter, plus the time you spend typing the examples yourself and changing them. Chapters 1 to 6 build the ideas, chapters 7 to 11 build the code, chapters 12 and 13 step back and look at consequences.

How the chapters work. Every chapter opens with a summary of the whole chapter, explains the concepts with tables and diagrams, shows numbered examples with the real output they produce, lists common mistakes with the best practice for each, and closes with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Every Python example in this volume was executed for real with Python 3.13 on Windows, and the output you see is the text that was printed. Save an example in a file such as `ch03_water_jug.py` and run it with `python ch03_water_jug.py`. Blocks tagged `python` are complete programs, blocks tagged `text` show what the program prints, and blocks tagged `bash` are terminal commands.

shell

```bash
python --version
python ch03_water_jug.py
```

output

```text
Python 3.13.15
start: jugs at (0, 0)
step 1: fill the 5-litre jug           -> jugs at (0, 5)
step 2: pour 5-litre into 3-litre      -> jugs at (3, 2)
step 3: empty the 3-litre jug          -> jugs at (0, 2)
step 4: pour 5-litre into 3-litre      -> jugs at (2, 0)
step 5: fill the 5-litre jug           -> jugs at (2, 5)
step 6: pour 5-litre into 3-litre      -> jugs at (3, 4)

solution length: 6 actions, 15 states visited
```

About the source. The syllabus follows the free course Elements of AI, published by MinnaLearn and the University of Helsinki. The topic names were used as a table of contents only: all explanations, examples, exercises and tests in this volume are original writing, and the course material itself was never copied.

A note on reproducibility. Every example that uses random numbers seeds the generator with `random.seed(42)` or with an explicit seed, so running the same script on your machine prints exactly the same lines that appear here.

Chapter map

#	Chapter	You will learn
1	What AI is (and is not)	definitions, related fields, rules versus learning, narrow and general intelligence
2	Philosophy of AI and the Turing test	the imitation game, the Chinese Room, what a passing test proves
3	Problem solving with search	states, graphs, breadth-first and depth-first search, the water jug puzzle
4	Game trees and minimax	game trees, minimax, alpha-beta pruning, a tic-tac-toe player
5	Probability, odds and Bayes' rule	probability, odds, likelihood ratios, the base rate fallacy
6	Naive Bayes classification in practice	a spam filter written from scratch, Laplace smoothing, leave-one-out
7	The types of machine learning	supervised, unsupervised and reinforcement learning, baselines
8	Nearest neighbours and regression	k-NN classification, linear regression, feature scaling
9	Neural network basics	the perceptron, layers, backpropagation by hand
10	Building and training a network	a two-layer network that learns XOR, seeds and learning rates
11	Practical AI with Python	an evaluation pipeline, confusion matrices, cross-validation
12	Ethics and the society AI works in	bias, fairness metrics, filter bubbles, anonymity, the EU AI Act
13	Where to go next	a twelve-week plan, self-assessment, keeping the habit
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - What AI is (and is not)

1.1 Summary

- Artificial intelligence is the study of programs that solve tasks we would call intelligent if a person did them.
- The same behaviour can be produced by rules a person typed in, or by rules a program derived from examples.
- Machine learning is the subfield that derives those rules from data instead of from a programmer.

- Deep learning, natural language processing, computer vision, robotics and search are branches that share methods with AI but are not identical to it.
- A system that beats the world champion at one game may still be unable to answer a simple question about another.
- AI is not a robot, not a database, and not magic: it is software that makes decisions from data.
- Writing rules by hand works when the rules are known and stable; learning works when examples are plentiful and rules are unclear.
- Modern systems usually combine both: hand-written structure around a learned core.
- Most practical failures come from bad data, unclear objectives, or a task that was never specified, not from clever machines.
- A useful habit is to ask, for every claim about AI, whether the claim is about behaviour or about understanding.

1.2 Concepts

Two ways to build the same behaviour

A program that labels e-mail as spam can be written in two very different ways.

Approach	Where the rules live	Good when	Weak when
Hand-written rules	in the programmer's head, then in `if` statements	the rule is short, stable and known	the rule is vague, long or changes
Learning from data	in numbers adjusted by a training procedure	many examples exist	data is scarce, biased or mislabelled

Neither approach understands the mail. Both produce a label. The difference is who wrote the rule: a person, or a loop that searched for numbers that fit the examples.

Two ways to make a decision, side by side

output

```
hand-written system                learning system
-------------------                ----------------

inputs                              inputs
  |                                   |
  v                                   v
+-----------+  rules written        +-----------+   weights adjusted
|  if rain  |  by a person          |  numbers  |   from examples
|   -> van  |  -------------------> |  in, out  |   ----------------->
+-----------+                       +-----------+
  |                                   |
  v                                   v
decision                           decision

easy to explain,                    hard to explain,
easy to break when                 improves as data grows
the world changes
```

Both columns are just code. The second one has more numbers and fewer words.

The AI family tree

AI is the umbrella; the other fields are branches that grew out of shared techniques.

Field	What it works on	Example task
Machine learning	tables of examples	predict a price from past sales
Deep learning	layered numeric models	recognise speech, generate text
Natural language processing	text and speech	translate a sentence
Computer vision	images and video	count items on a shelf
Search and planning	states and actions	find a route, solve a puzzle
Robotics	sensors and actuators	move an arm to a target
Knowledge representation	facts and rules	answer questions over a rule base

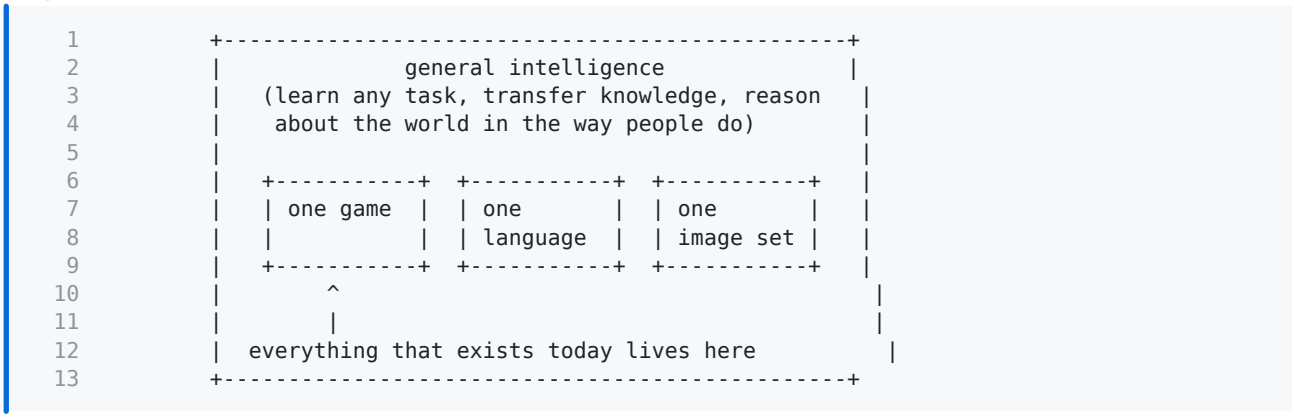
What AI is not

Common claim	What is actually true
AI means humanoid robots	most AI runs on servers and has no body at all
AI understands what it processes	it computes outputs; understanding is a separate philosophical question
AI is always right	it is right on average, and can be confidently wrong
AI replaces the whole job	it usually replaces one step inside a job
AI needs consciousness to be useful	no useful system today is conscious as far as anyone knows
More data always helps	better matched data helps; mismatched data just costs money

Narrow and general

Today's systems are narrow: excellent inside one boundary, helpless outside it.

output



The boxes near the bottom are narrow systems. The space around them is what people usually mean by "AI" in conversation, and it is mostly still empty.

1.3 Examples

Example 1: a rule system written entirely by hand, the knowledge-engineering way. Nothing is learned; a person decided every threshold.

Python

```python
"""A rule system written entirely by hand: the "knowledge engineering" way."""

income = 32000          # euros per year
years_employed = 3
late_payments = 0
requested = 12000       # euro loan requested

score = 0
score += 40 if income >= 24000 else 0
score += 30 if years_employed >= 2 else 0
score -= 25 if late_payments > 0 else 0
score += 20 if requested <= income / 2 else 0

decision = "approve" if score >= 60 else ("review" if score >= 40 else "decline")

print("income          %7d EUR" % income)
print("years employed  %7d" % years_employed)
print("late payments   %7d" % late_payments)
print("requested loan  %7d EUR" % requested)
print("-" * 24)
print("score           %7d" % score)
print("decision        %s" % decision.upper())
```

output

```
income            32000 EUR
years employed        3
late payments         0
requested loan    12000 EUR
------------------------
score                90
decision        APPROVE
```

The score is 40 + 30 + 20: income is high enough, employment is long enough, the loan is small enough, and there are no late payments. Every number was chosen by a person, so the program cannot improve and cannot explain why the threshold 24000 is the right one.

Example 2: the same idea expressed as a small decision tree, then checked against eight labelled deliveries.

Python

```
"""A decision tree written by hand - no learning, only rules a person typed in."""

"""rows: (rain, wind_kmh, parcel_kg, label the person would choose)"""
deliveries = [
    (False,  8, 1.2, "drone"),
    (False, 30, 0.8, "van"),
    (True,   5, 0.5, "van"),
    (False, 12, 6.0, "van"),
    (False, 15, 2.4, "drone"),
    (False, 40, 1.1, "van"),
    (False, 10, 3.9, "drone"),
    (True,  18, 2.0, "van"),
]

def tree(rain, wind, kg):
    if kg > 5.0:
        return "van"      # too heavy to fly
    if rain:
        return "van"      # rain stops the drone
    if wind > 25:
        return "van"      # wind over 25 km/h stops the drone
    return "drone"

correct = 0
for rain, wind, kg, label in deliveries:
    guess = tree(rain, wind, kg)
    ok = guess == label
    correct += ok
    print("rain=%-5s wind=%2d km/h kg=%4.1f -> %-5s (person: %-5s) %s"
          % (rain, wind, kg, guess, label, "ok" if ok else "WRONG"))

print()
print("accuracy: %d/%d = %.0f%%" % (correct, len(deliveries),
                                    100.0 * correct / len(deliveries)))
```

output

```
rain=False wind= 8 km/h kg= 1.2 -> drone (person: drone) ok
rain=False wind=30 km/h kg= 0.8 -> van   (person: van  ) ok
rain=True  wind= 5 km/h kg= 0.5 -> van   (person: van  ) ok
rain=False wind=12 km/h kg= 6.0 -> van   (person: van  ) ok
rain=False wind=15 km/h kg= 2.4 -> drone (person: drone) ok
rain=False wind=40 km/h kg= 1.1 -> van   (person: van  ) ok
rain=False wind=10 km/h kg= 3.9 -> drone (person: drone) ok
rain=True  wind=18 km/h kg= 2.0 -> van   (person: van  ) ok

accuracy: 8/8 = 100%
```

This tree is small, readable and perfect on these eight rows. It is also brittle: a parcel of 5.5 kg in a 24 km/h wind with light rain gets a decision no one ever discussed.

Example 3: how the programs in this volume are run.

shell

```
python --version
python ch01_decision_tree.py
```

output

```
Python 3.13.15
rain=False wind= 8 km/h kg= 1.2 -> drone (person: drone) ok
rain=False wind=30 km/h kg= 0.8 -> van   (person: van  ) ok
rain=True  wind= 5 km/h kg= 0.5 -> van   (person: van  ) ok
rain=False wind=12 km/h kg= 6.0 -> van   (person: van  ) ok
rain=False wind=15 km/h kg= 2.4 -> drone (person: drone) ok
rain=False wind=40 km/h kg= 1.1 -> van   (person: van  ) ok
rain=False wind=10 km/h kg= 3.9 -> drone (person: drone) ok
rain=True  wind=18 km/h kg= 2.0 -> van   (person: van  ) ok

accuracy: 8/8 = 100%
```

The first command only prints the interpreter version; the second runs the file you saved. Nothing is installed and no library is imported in this volume.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Calling any clever-looking behaviour "AI"	the word stops carrying information	name the technique: rules, search, learning, lookup
Expecting a rule system to improve itself	hand-written rules never change on their own	feed it new rules deliberately, or move to learning
Choosing thresholds with no reason	the boundaries look arbitrary and hide value judgements	record why each threshold exists, and who chose it
Judging a model on the rows it was built from	the score only measures memory	hold data aside and test on unseen rows
Assuming a perfect score means a solved problem	eight friendly rows prove very little	look for the hardest cases, then for cases nobody collected
Treating a wrong answer as proof the machine is stupid	one error says nothing about the method	measure error rates, then inspect the failures

- When someone says "the AI decided", ask which inputs were used and which rule or numbers turned them into the output.
- Any system described in words you can copy into `if` statements is a rule system, whatever the marketing says.

1.5 Practice exercises

1. A vending machine accepts a card, checks the balance and dispenses a drink. Is this AI? Justify your answer using the definitions in this chapter.
2. Write, by hand and without code, a rule system of at most five rules that decides whether a taxi should pick up a passenger, and name the information your rules need.
3. The hand-written tree in Example 2 gets every row right. Give two reasons this does not prove the tree is good.
4. Name two situations where hand-written rules beat learning, and two where learning beats hand-written rules.

5. For each of these systems, say which field it belongs to: a spam filter, a parking sensor that steers a car, a chatbot that answers school questions about its own rules, a program that finds the shortest route in a city.

Solutions

1. Under the definition used here, no: every step is a lookup and a comparison that a person can write down in advance. It is a perfectly programmed machine, but no part of it was derived from examples or searched for.
2. Example answer: (1) the passenger has a confirmed booking, (2) the pickup address is inside the operating area, (3) the driver is not already on a trip, (4) the car has enough battery or fuel, (5) no safety flag on the account. The rules need the booking record, the map boundary, the driver state, the fuel level and the account flag.
3. First, the tree was read off those same rows, so the score measures agreement, not generalisation. Second, eight rows cannot cover the corners: wet but light parcels, warm but gusty days, or a drone with a shorter range than assumed.
4. Rules win when the rule is short and stable (tax bands, a board game's legal moves, a safety interlock) and when wrong answers are expensive and an explanation is required. Learning wins when the pattern is unclear (which e-mail is spam, which photo contains a crack) and when many examples are cheaply available.
5. Spam filter: machine learning. Parking and steering: robotics and computer vision. Question answering over its own rules: search, knowledge representation or rule following. Shortest route: search and planning.

1.6 Mini test

1. What does this volume mean by artificial intelligence? A) Any program that runs on a server B) Software that performs tasks we would call intelligent if a person did them C) A robot with a human shape D) A database large enough to hold the internet
2. Where do the rules of a learning system come from? A) A programmer writes them all B) A procedure adjusts numbers so that examples are fitted C) They are copied from a textbook D) They are drawn at random and never change
3. Which statement about narrow systems is true? A) They can transfer skill between any two tasks B) They work excellently inside one boundary and poorly outside it C) They are general once they pass enough tests D) They always understand what they process
4. True or false: a hand-written rule system can become better without anyone editing its rules.
5. Which is the best first question to ask when someone claims a system is "AI"? A) Which company built it B) What data or rules produce its output C) How many computers it uses D) Whether it has a logo
6. The decision tree in Example 2 scores 100 percent on its own data. What is the correct conclusion? A) The tree solves the general delivery problem B) The tree agrees with those eight rows and must now be tested elsewhere C) The tree has learned from data D) The tree's thresholds are provably correct

Answer key and explanations

1. B - AI is defined by the kind of task, not by hardware, size or shape.
2. B - in machine learning the "rules" are numbers adjusted by a training procedure driven by examples.
3. B - modern systems are narrow; they excel in a boundary and fail outside it, which is why a chess champion program cannot answer weather questions.
4. False - hand-written behaviour only changes when a person edits it; that is precisely the difference from learning.
5. B - behaviour can come from rules or from learned numbers, so the inputs and the mechanism are what matter.
6. B - performance on the rows used to build the model measures agreement, not generalisation; unseen data is the real test.

Chapter 2 - Philosophy of AI and the Turing test

2.1 Summary

- The Turing test asks a behavioural question: can a machine hold a conversation so well that a judge cannot tell it from a person?
- Alan Turing proposed it in 1950 to replace the unanswerable question "can machines think" with one that can be tested.
- The test runs over a text channel, so it deliberately ignores voice, face and body.
- The Chinese Room argument, by John Searle, claims that correct symbol handling can look exactly like understanding while containing none.
- A judge is a fallible detector: simple cues such as short answers or hedging words can decide the verdict, as the example below shows.
- Weak artificial intelligence aims at useful behaviour; strong artificial intelligence aims at a genuine mind.
- Philosophical arguments rarely change what engineers build, but they change what is required, what is promised, and who is responsible.
- Claims about machine understanding are untestable in a lab, which is why specifications speak about behaviour.
- A good habit is to separate two questions that are constantly confused: what does the system do, and what does the system mean.

2.2 Concepts

The imitation game

The original setting has three participants and one restriction.

Participant	Role	Sees
Interrogator	asks questions, guesses who is who	typed answers only
Human	answers as themselves	everything
Machine	imitates a human	typed answers only

output

```
+--------------+          questions          +----------------+
| interrogator |  ------------------------> |     human      |
|   (judge)    | <------------------------- |   answerer     |
+--------------+          answers           +----------------+
       |  ^
       |  |
       v  |
+--------------+
|   machine    |   same questions, same kind of answers
|  answerer    |   goal: judge cannot pick it out reliably
+--------------+
```

If the judge cannot do better than chance over many trials, the machine has passed. Note what is measured: text that a person finds convincing.

What the test can and cannot settle

Question	Can the imitation game decide it?
Can this system produce text that reads as human?	yes
Does this system follow rules or produce statistics?	no
Does this system understand the conversation?	no, not by design
Is this system safe to deploy in a hospital?	no, that needs separate evaluation
Is this system intelligent in a general sense?	no; it says nothing about other tasks

The Chinese Room

Searle's thought experiment puts a person who speaks no Chinese in a room with a rulebook that maps incoming Chinese symbols to outgoing ones. From outside, the answers look fluent; inside, no understanding is involved.

output

```
   slot in the wall                  rule book                    slot out
+------------------+        +--------------------------------+   +-----------------+
|  shape "rain-1"  | -----> | find "rain-1", look up its     | ->|  shape "sun-3"  |
|  shape "sky-2"   |        | paired reply, copy it onto a   |   |  shape "cat-7"  |
|  ...             |        | slip and push it out           |   |  ...            |
+------------------+        +--------------------------------+   +-----------------+

outside observer: "the room understands Chinese"
inside clerk    : "these are only shapes; the book does the work"
```

The argument is about syntax versus semantics: correct handling of symbols does not automatically produce meaning. Replies to the argument are numerous, which is itself the

interesting point: the question is not settled.

Weak and strong claims

Claim type	Statement	How it is judged
Weak (engineering)	this system classifies e-mail at a measured error rate	experiments and metrics
Strong (philosophical)	this system has understanding or a mind	argument, not experiment
Popular	this system is smarter than people	usually one narrow task, badly defined

2.3 Examples

Example 1: the Chinese Room as a small simulation. A dictionary stands in for the rule book, and a loop stands in for the clerk.

Python

```python
"""The Chinese Room, as a tiny simulation: symbols in, symbols out."""

rulebook = {
    "ni hau ma": "wo hen hao",
    "tianqi zenmeyang": "jintian xia yu",
    "ni zuo shenme gongzuo": "wo shi yi ming laoshi",
}

questions = ["ni hau ma", "tianqi zenmeyang", "ni zuo shenme gongzuo"]

for q in questions:
    print("outside   :", q)
    print("received  :", q, "(an unlabelled string of symbols)")
    print("rule used : look up the first symbol, reply with the paired one")
    print("returned  :", rulebook[q])
    print()

print("The clerk inside never learns the language; only shapes are shuffled.")
```

output

```
outside   : ni hau ma
received  : ni hau ma (an unlabelled string of symbols)
rule used : look up the first symbol, reply with the paired one
returned  : wo hen hao

outside   : tianqi zenmeyang
received  : tianqi zenmeyang (an unlabelled string of symbols)
rule used : look up the first symbol, reply with the paired one
returned  : jintian xia yu

outside   : ni zuo shenme gongzuo
received  : ni zuo shenme gongzuo (an unlabelled string of symbols)
rule used : look up the first symbol, reply with the paired one
returned  : wo shi yi ming laoshi

The clerk inside never learns the language; only shapes are shuffled.
```

The replies are fluent enough for this script, and nothing is understood: the program only matches keys. Ask a question that is not a key, for example `ni ji sui`, and the program crashes with a `KeyError` rather than answering. Understanding would show up as graceful handling of the unlisted case.

Example 2: a miniature imitation game with a judge that cannot see the two answerers. The judge is deliberately shallow: it looks at word count and hedging words.

Python

```python
"""A miniature imitation game with a judge that cannot see the answerers."""

machine = [
    "I like long walks and clean code.",
    "I grew up in a small town, I think?",
    "My favourite food is pasta, definitely.",
]

human = [
    "I grew up in Tallinn, near the sea.",
    "My sister teaches maths there.",
    "I prefer rye bread, honestly.",
]

def judge_guess(answer):
    """A shallow judge: word count plus a list of hedging words."""
    hedges = ("actually", "honestly", "definitely", "i think", "near", "there")
    score = 0
    if len(answer.split()) < 8:
        score += 1
    if any(h in answer for h in hedges):
        score += 1
    return "human" if score >= 1 else "machine"

questions = [
    "What do you enjoy doing?",
    "Where did you grow up?",
    "What is your favourite food?",
]

wrong = 0
for i in range(3):
    verdict = judge_guess(machine[i])
    mistaken = verdict == "human"
    wrong += mistaken
    print("Q%d: %s" % (i + 1, questions[i]))
    print("    machine answers: %s" % machine[i])
    print("    judge verdict  : %s %s" % (verdict, "(wrong!)" if mistaken else ""))
    print("    human answers  : %s" % human[i])
    print()

print("The judge called the machine 'human' %d times out of 3." % wrong)
print("A test that everyone passes tells us very little.")
```

output

```
Q1: What do you enjoy doing?
    machine answers: I like long walks and clean code.
    judge verdict  : human (wrong!)
    human answers  : I grew up in Tallinn, near the sea.

Q2: Where did you grow up?
    machine answers: I grew up in a small town, I think?
    judge verdict  : machine
    human answers  : My sister teaches maths there.

Q3: What is your favourite food?
    machine answers: My favourite food is pasta, definitely.
    judge verdict  : human (wrong!)
    human answers  : I prefer rye bread, honestly.

The judge called the machine 'human' 2 times out of 3.
A test that everyone passes tells us very little.
```

The machine has no model of a person, no memory of the conversation and no beliefs. It passes twice because its echoes are short and full of hedging words, and because the judge's rule is exactly that shallow. A test is only as informative as the detector behind it.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating the Turing test as a definition of thinking	it measures one text channel, nothing more	say "passes an imitation game" instead of "thinks"
Reading a friendly answer as evidence of a mind	fluent text is produced by statistical pattern matching	judge systems by measured behaviour on a task
Confusing syntax with semantics	correct symbol handling is not understanding	keep the two claims separate in writing and in tests
Designing a judge from a single cue	one cue decides everything and the test proves nothing	make the judge adversarial and measure its accuracy on its own
Using philosophy to settle engineering questions	arguments about minds do not pick a model	settle measurable questions with experiments
Claiming that a system "knows" what it never saw	hallucination becomes indistinguishable from knowledge	evaluate on held-out cases and inspect failures

- Whenever a system produces text a person finds convincing, ask which mechanism produced it: a rule, a lookup, or a fitted model.
- The engineering value of the thought experiment is that it forces specifications to talk about behaviour, which is testable.

2.5 Practice exercises

1. State the Turing test in your own words in at most three sentences, then say what it deliberately leaves out.

2. In the Chinese Room, who or what follows a rule, and what would have to be added for the room to genuinely understand?
3. The judge in Example 2 is fooled twice. Give a concrete change to the judge that would make the same three answers fail.
4. A colleague says "the chatbot passed the Turing test, so it is conscious". Rewrite the claim so that it says only what is supported.
5. Which is easier to test: "the system classifies 90 percent of spam correctly" or "the system understands language"? Why?

Solutions

1. A judge holds typed conversations with a hidden human and a hidden machine and tries to tell them apart; if the judge cannot reliably pick out the machine, the machine passes. It leaves out everything but text: intonation, actions, and whatever happens inside either answerer.
2. The rulebook and the clerk do the work; the clerk follows instructions without knowing what the symbols mean. What would have to be added is exactly what is in dispute: meaning, intention or grounded experience, and no proposed addition has been accepted as settling the question.
3. Ask something that requires memory of an earlier answer, something that needs current world knowledge, or a question that only a person with a body could answer; the canned replies repeat the same hedging patterns and cannot follow a thread.
4. "In three scripted questions the system produced answers that a keyword judge classified as human twice; no claim about understanding is implied, and the judge's accuracy was not measured on other answers."
5. The spam claim, because it refers to a labelled set and a measurable error rate; "understands language" has no agreed measurement, so no experiment settles it.

2.6 Mini test

1. What exactly does the Turing test measure? A) Whether a machine has consciousness B) Whether a judge can distinguish machine text from human text C) Whether a machine can beat a person at chess D) Whether a machine can move like a person
2. Why did Turing phrase the question as an imitation game? A) Because machines cannot play games B) Because "can machines think" is too vague to test, while the game can be run C) Because he wanted a television contest D) Because text is easier for machines than arithmetic
3. What is the Chinese Room argument trying to show? A) That Chinese is too hard for computers B) That manipulating symbols correctly can look like understanding without any understanding C) That rule books are better than statistics D) That rooms cannot contain people
4. True or false: a system that passes a shallow test has been shown to be generally intelligent.

5. Which claim is a weak, testable claim? A) The system knows what the user means B) The system answers 80 percent of the questions in a fixed test set without inventing facts C) The system has beliefs D) The system deserves rights
6. In Example 2, why is the judge's verdict weak evidence? A) Because the machine was written by a person B) Because the judge only checks word count and hedging words, so a rule could pass C) Because three questions are always enough D) Because the human answers were too short

Answer key and explanations

1. B - the test compares behaviour through a typed channel, nothing about the machine's inner state.
2. B - Turing replaced an unanswerable metaphysical question with an operational one that can be run and scored.
3. B - the room produces correct outputs without any comprehension of the symbols, which is the point of the argument.
4. False - passing one narrow channel test says nothing about other tasks, and a bad judge can be fooled by a rule.
5. B - it refers to a fixed set, a threshold and a definition of inventing facts, so it can be measured.
6. B - a detector with one shallow cue cannot separate a real answerer from a template; the judge must be evaluated itself.

Chapter 3 - Problem solving with search

3.1 Summary

- A large share of classic AI work is phrased as search: find a path from a start state to a goal state.
- A state is a complete description of a situation, and an action turns one state into the next.
- The state space is every state reachable by applying actions; the search walks through it.
- Breadth-first search expands level by level, so it returns a route with the fewest steps.
- Depth-first search dives as deep as it can first, which saves memory but can return a long detour.
- Iterative deepening runs depth-first searches with growing depth limits, combining optimality with small memory.
- When roads have different lengths, Dijkstra's method searches in order of distance rather than of hop count.
- The way the problem is represented decides what the search can even see, and how big the space is.
- Search powers route planners, puzzle solvers, automatic planners and the move choice in games.

- In the examples, every queue, every visited set and every path is printed, so nothing is hidden.

3.2 Concepts

States, actions and the state space

Term	Meaning	In the jug puzzle
State	full description at one moment	litres in each jug, such as `(2, 5)`
Action	a move that changes the state	fill, empty, pour
State space	all states reachable from the start	every pair `(a, b)` reachable from `(0, 0)`
Start	where the search begins	`(0, 0)`
Goal	states that stop the search	any state whose second value is 4
Path	the sequence of actions that got there	six actions, printed by the program

Level by level, or head first?

Method	Order of expansion	Path found	Memory	Fails when
Breadth-first	all states one step away, then two, ...	fewest steps	stores a whole level	the frontier is huge
Depth-first	one branch as deep as possible, then backtrack	whatever branch it picked	stores one branch	the branch is a dead end
Iterative deepening	depth 0, then 1, then 2, ...	fewest steps	stores one branch	the branching factor is very large, so levels are re-walked
Uniform cost (Dijkstra)	cheapest total distance first	cheapest distance	stores all open states	all distances are 1, where it equals breadth-first

output

```
   A
  / \
 B   C
 |   |
 D   E
 |   |
 G   F      goal G

breadth-first : A, B, C, D, E, F, G      fewest steps, wide
depth-first   : A, B, D, G               deep first, narrow
iterative     : A | A,B,C | A,B,C,D,E,F,G  same answer, repeated work
```

Why the representation matters

Two programs searching for the same answer can differ by millions of steps, because of how the situation is described.

Choice	Effect
store only what the goal needs	smaller states, fewer duplicates
treat the order of a list as irrelevant	far fewer states for puzzles with interchangeable pieces
forbid illegal actions before they are generated	the search never wastes a level on them
merge states that are really the same	memory stays proportional to real situations

3.3 Examples

Example 1: breadth-first search and Dijkstra's method on one small map of seven towns. The program prints the layers as they appear, then the route with the fewest hops, then the route with the shortest distance.

Python

```python
"""Breadth-first search and Dijkstra on the same small map."""

graph = {
    "Harbor":  [("Forge", 3), ("Mill", 4)],
    "Mill":    [("Harbor", 4), ("Forge", 2), ("Quarry", 5)],
    "Forge":   [("Harbor", 3), ("Mill", 2), ("Ridge", 7)],
    "Quarry":  [("Mill", 5), ("Cistern", 4)],
    "Ridge":   [("Forge", 7), ("Cistern", 2), ("Bastion", 6)],
    "Cistern": [("Quarry", 4), ("Ridge", 2), ("Bastion", 3)],
    "Bastion": [("Ridge", 6), ("Cistern", 3)],
}

def bfs(start, goal):
    seen = {start}
    parent = {start: None}
    layer = [start]
    print("layer 0: %s" % start)
    step = 0
    while layer:
        step += 1
        nxt = []
        for town in layer:
            for name, _km in graph[town]:
                if name not in seen:
                    seen.add(name)
                    parent[name] = town
                    nxt.append(name)
        if nxt:
            print("layer %d: %s" % (step, " ".join(sorted(nxt))))
        if goal in seen:
            break
        layer = nxt
    path = [goal]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return list(reversed(path))

def dijkstra(start, goal):
    dist = {town: 10 ** 9 for town in graph}
    dist[start] = 0
    parent = {start: None}
    done = set()
    while len(done) < len(graph):
        town = min((t for t in graph if t not in done), key=lambda t: dist[t])
        done.add(town)
        for name, km in graph[town]:
            if dist[town] + km < dist[name]:
                dist[name] = dist[town] + km
                parent[name] = town
    path = [goal]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return list(reversed(path)), dist[goal]

print("graph: 7 towns, roads with distances in km")
print()
```

Python (continued)

```
path = bfs("Harbor", "Bastion")
print()
print("fewest hops:", " -> ".join(path), "(%d hops)" % (len(path) - 1))
print()
best, km = dijkstra("Harbor", "Bastion")
print("shortest distance: %s (%d km)" % (" -> ".join(best), km))
```

output

```
graph: 7 towns, roads with distances in km

layer 0: Harbor
layer 1: Forge Mill
layer 2: Quarry Ridge
layer 3: Bastion Cistern

fewest hops: Harbor -> Forge -> Ridge -> Bastion (3 hops)

shortest distance: Harbor -> Forge -> Ridge -> Cistern -> Bastion (15 km)
```

The two answers differ, and both are correct. Searching by hops favours the direct but long road over the mountain shortcut plus two short roads: 3 hops against 4 hops, but 16 km against 15 km. The layers also show why breadth-first search is called level by level: Harbor, then everything one road away, then everything two roads away.

Example 2: a state-space search that most people only solve with pen and paper. Two jugs of three and five litres, no markings, and the target is exactly four litres.

Python

```python
"""A state-space search: measure exactly 4 litres using a 3-litre and a
5-litre jug, with no markings and no other container."""

from collections import deque

CAP_A, CAP_B = 3, 5
GOAL = 4

def successors(state):
    a, b = state
    result = [
        ((CAP_A, b), "fill the 3-litre jug"),
        ((a, CAP_B), "fill the 5-litre jug"),
        ((0, b), "empty the 3-litre jug"),
        ((a, 0), "empty the 5-litre jug"),
    ]
    pour = min(a, CAP_B - b)
    result.append(((a - pour, b + pour), "pour 3-litre into 5-litre"))
    pour = min(b, CAP_A - a)
    result.append(((a + pour, b - pour), "pour 5-litre into 3-litre"))
    return result

start = (0, 0)
seen = {start}
parent = {start: None}
action = {}
queue = deque([start])
goal_state = None
while queue:
    state = queue.popleft()
    if state[1] == GOAL:
        goal_state = state
        break
    for nxt, act in successors(state):
        if nxt not in seen:
            seen.add(nxt)
            parent[nxt] = state
            action[nxt] = act
            queue.append(nxt)

path = [goal_state]
while parent[path[-1]] is not None:
    path.append(parent[path[-1]])
path.reverse()

print("start: jugs at %s" % (path[0],))
for step, state in enumerate(path[1:], 1):
    print("step %d: %-30s -> jugs at %s" % (step, action[state], state))
print()
print("solution length: %d actions, %d states visited"
      % (len(path) - 1, len(seen)))
```

output

```
start: jugs at (0, 0)
step 1: fill the 5-litre jug           -> jugs at (0, 5)
step 2: pour 5-litre into 3-litre      -> jugs at (3, 2)
step 3: empty the 3-litre jug          -> jugs at (0, 2)
step 4: pour 5-litre into 3-litre      -> jugs at (2, 0)
step 5: fill the 5-litre jug           -> jugs at (2, 5)
step 6: pour 5-litre into 3-litre      -> jugs at (3, 4)

solution length: 6 actions, 15 states visited
```

The search never plans; it remembers. `parent` stores where each state came from, `seen` prevents a second visit, and the queue decides what to open next. Because the queue is first-in first-out, the first goal reached is reached by the fewest actions. Only 15 of the 24 possible states are needed.

Example 3: three search strategies on the same graph, counting the states they open. The graph is built so that the long branch is visited first.

Python

```python
"""Depth-first, breadth-first and iterative deepening search on one graph."""

from collections import deque

graph = {
    "A": ["C", "B"],
    "B": ["G"],
    "C": ["D"],
    "D": ["E"],
    "E": ["F"],
    "F": ["G"],
    "G": [],
}

def dfs(start, goal):
    stack = [(start, [start])]
    expanded = 0
    while stack:
        node, path = stack.pop()
        expanded += 1
        if node == goal:
            return path, expanded
        for nxt in reversed(graph[node]):
            if nxt not in path:
                stack.append((nxt, path + [nxt]))
    return None, expanded

def bfs(start, goal):
    queue = deque([(start, [start])])
    expanded = 0
    while queue:
        node, path = queue.popleft()
        expanded += 1
        if node == goal:
            return path, expanded
        for nxt in graph[node]:
            if nxt not in path:
                queue.append((nxt, path + [nxt]))
    return None, expanded

def limited_dfs(start, goal, limit):
    stack = [(start, [start], 0)]
    expanded = 0
    while stack:
        node, path, depth = stack.pop()
        if node == goal:
            return path, expanded
        expanded += 1
        if depth < limit:
            for nxt in reversed(graph[node]):
                if nxt not in path:
                    stack.append((nxt, path + [nxt], depth + 1))
    return None, expanded

def iddfs(start, goal, max_depth=10):
    total = 0
```

Python (continued)

```
    for limit in range(max_depth + 1):
        found, nodes = limited_dfs(start, goal, limit)
        total += nodes
        if found:
            return found, total
    return None, total

d_path, d_nodes = dfs("A", "G")
b_path, b_nodes = bfs("A", "G")
i_path, i_nodes = iddfs("A", "G")

print("%-26s %-18s %-6s %s" % ("method", "path", "hops", "nodes expanded"))
print("-" * 66)
print("%-26s %-18s %-6d %d" % ("depth-first", "-".join(d_path),
                               len(d_path) - 1, d_nodes))
print("%-26s %-18s %-6d %d" % ("breadth-first", "-".join(b_path),
                               len(b_path) - 1, b_nodes))
print("%-26s %-18s %-6d %d" % ("iterative deepening", "-".join(i_path),
                               len(i_path) - 1, i_nodes))
print()
print("the graph: A-C-D-E-F-G is the long way, A-B-G the short one")
```

output

```
method                     path               hops   nodes expanded
------------------------------------------------------------------
depth-first                A-C-D-E-F-G        5      6
breadth-first              A-B-G              2      5
iterative deepening        A-B-G              2      8

the graph: A-C-D-E-F-G is the long way, A-B-G the short one
```

Depth-first found a path in six expansions but a poor one, five hops along the spine of the graph. Breadth-first found the two-hop shortcut in five expansions. Iterative deepening found the same shortcut with eight expansions because it starts the depth-first walk again for each limit, the price it pays for using the memory of only one branch.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Expanding states that were already seen	the search can loop forever	keep a visited set and skip repeats
Storing full paths in the queue	memory blows up on large spaces	store a parent pointer and rebuild the path
Mixing up hops and distance	"shortest" quietly changes meaning	say which one you mean, and use Dijkstra for weighted graphs
Generating illegal actions and rejecting them later	wasted work in every level	build the successor function so that only legal moves appear
Using depth-first search when the shortest path matters	it can return a long detour	use breadth-first, iterative deepening or uniform cost
Ignoring the size of the state space	the search never finishes and no one notices	print the expanded count and the frontier size while it runs

- Print the first ten expansions of any new search before trusting it.
- If two states describe the same situation in different clothing, teach the successor function to see them as one state.

3.5 Practice exercises

1. Describe the state, the actions, the start, the goal and the state space for a puzzle where a farmer must move a fox, a chicken and grain across a river.
2. Why does breadth-first search need more memory than depth-first search on a graph with many branches?
3. In Example 2, the program visits 15 states. How many states exist in total for two jugs of three and five litres, and why is the search smaller than that?
4. A courier company optimises driving time, not the number of turns. Which search from this chapter fits, and what must change in the code?
5. The depth-first search in Example 3 returns a five-hop path. Change one line of the graph so that it returns a three-hop path, and say which one.

Solutions

1. State: which side of the river each of the farmer, fox, chicken and grain is on. Actions: move the farmer alone, or with one item, provided the fox and chicken are never left alone together, nor the chicken and the grain. Start: everything on the near bank. Goal: everything on the far bank. State space: the legal combinations of the four positions.
2. Breadth-first search keeps an entire level in the queue while it expands the next one, and the level grows with the branching factor, while depth-first keeps only one branch plus the unexplored siblings along it.
3. There are 4 times 6, or 24, states, since the three-litre jug holds 0 to 3 litres and the five-litre jug 0 to 5. The search visits fewer because pouring always fills one jug or empties the other, so states such as `(1, 1)` are unreachable from `(0, 0)`.
4. Uniform cost search, which is Dijkstra's method: expand the open state with the smallest total driving time. The change is to store a distance with each state and to pick the next state by distance instead of by position in a queue.
5. Give `A` the neighbour order `["B", "C"]`, so the last branch of the depth-first stack is the one that leads to `G` in two hops; the path then becomes `A-B-G`.

3.6 Mini test

1. Which statement describes a state? A) One action the program may take B) A complete description of the situation at one moment C) The route from start to goal D) The rule that stops the search
2. What does breadth-first search guarantee? A) The cheapest route in kilometres B) A route with the fewest actions C) A route that uses the least memory D) Any route at all, if one exists

3. Which method re-runs depth-first search with growing depth limits? A) Uniform cost search B) Iterative deepening C) Dijkstra's method D) Backtracking without limits
4. True or false: in a depth-first search the first goal found is always reached by the shortest route.
5. What does the `seen` set prevent in the water jug program? A) Illegal actions B) The same state being expanded twice C) The queue from growing too small D) Goals that are not exactly four litres
6. Which change turns the hop-count search of Example 1 into a distance search? A) Sort the neighbours alphabetically B) Replace the queue with a cost table and always expand the cheapest open state C) Stop when the goal is first seen instead of when it is expanded D) Reverse the parent pointers at the end

Answer key and explanations

1. B - a state carries everything needed to generate successors and to test the goal; actions, paths and stopping rules are separate objects.
2. B - expanding level by level means no shorter route can be waiting behind a longer one, though the route need not be the cheapest in distance.
3. B - iterative deepening repeats depth-first searches with limits 0, 1, 2 and so on until a goal appears.
4. False - depth-first follows one branch, so the first goal can be far away along a detour, as the five-hop path in Example 3 shows.
5. B - `seen` records expanded states so that the queue never receives a duplicate, which also stops loops in cyclic graphs.
6. B - distances need a cost order rather than a queue order, which is exactly what uniform cost, or Dijkstra's method, does.

Chapter 4 - Game trees and minimax

4.1 Summary

- A two-player game with perfect information is a search problem where the opponent has the opposite objective.
- A game tree has one node per position and one child per legal move, and its leaves receive values.
- The maximiser picks the child with the largest value, the minimiser the child with the smallest value.
- Minimax backs values up from the leaves, assuming both sides choose their best reply at every level.
- Scores can include the number of moves left, so a win in one move beats a win in ten.
- Alpha-beta pruning skips branches that cannot change the value already found, and returns exactly the same answer.

- When the search stops before the end of the game, an evaluation function must estimate the position.
- Games with chance need expected values over the possible outcomes, not only the best outcome.
- The same alternating recursion appears outside games, for example in contract negotiation and in safety checks.
- Tic-tac-toe is small enough to search completely, which makes it a laboratory for counting and pruning.

4.2 Concepts

The vocabulary of a game tree

Term	Meaning	In tic-tac-toe
Root	the position to decide	the board at the start of a turn
Ply	one level of the tree, one move deep	one player's move
Leaf	a terminal position	a win, a loss or a full board
Value	the score backed up to the root	positive for the maximiser
Best move	the child whose value equals the root value	the cell the program plays

Backing values up a tree

The tree below is the one in Example 1. Levels alternate between a maximiser and a minimiser, and the leaf values are read off the bottom.

output

```
                    [ max ]                      the maximiser chooses a branch
                   /       \
              [ min ]         [ min ]            each minimiser picks its worst
             /       \       /       \
          [ max ]  [ max ]  [ max ]  [ max ]       max over two min values
          / \      / \      / \      / \
         3  12   8   2    2   9   11   3   7   1   10   1
         |__|    |__|     |__|    |__|
          3      8        3      4                    <- the min levels
         |_____________|  |_____________|
               3               1                     <- root value 3
```

Reading the tree bottom-up: each maximum takes the larger of its two children, each minimum takes the smaller, and the root takes the larger of the two branch values. The left branch is worth 3 and the right branch 1, so the maximiser plays left and the position is worth 3.

Minimax and alpha-beta

Property	Plain minimax	With alpha-beta pruning
Value at the root	exact	identical
Branches examined	all of them	only those that can matter
Speed	fine for small trees	the same answer with far fewer nodes
Risk	none	none, if the update rules are followed exactly
When pruning happens	never	as soon as a player can already do better

Two numbers, `alpha` for the best the maximiser can guarantee and `beta` for the best the minimiser can guarantee, travel down the tree. When `beta <= alpha` in a node, the remaining children cannot change the result, so they are skipped.

When the search must stop early

In chess-like games the tree is far too large, so the program searches a fixed number of plys and estimates the position with an evaluation function.

Approach	Needs	Risk
Search to the end	a small game, such as tic-tac-toe	none
Search to a depth limit	an evaluation function	the estimate can be wrong, and the mistake hides below the horizon
Search with quiescence	a quiet-position test	extra time for the same idea

4.3 Examples

Example 1: minimax on a hand-built tree, with and without pruning. The two answers must agree, and the branch values are printed so the recursion can be checked by hand.

Python

```python
"""Minimax with and without alpha-beta on a small game tree.
The root is a maximising node; levels alternate min, max, then leaves."""

tree = (
    (
        ((3, 12), (8, 2)),
        ((4, 6), (14, 5)),
    ),
    (
        ((2, 9), (11, 3)),
        ((7, 1), (10, 1)),
    ),
)

def is_leaf(node):
    return isinstance(node, (int, float))

stats = {"plain": 0, "pruned": 0}

def minimax(node, depth, alpha, beta, use_pruning):
    if is_leaf(node):
        return node
    if use_pruning:
        stats["pruned"] += 1
    else:
        stats["plain"] += 1
    if depth % 2 == 0:                      # maximising player
        best = -999
        for child in node:
            best = max(best, minimax(child, depth + 1, alpha, beta, use_pruning))
            alpha = max(alpha, best)
            if use_pruning and beta <= alpha:
                break
        return best
    best = 999                              # minimising player
    for child in node:
        best = min(best, minimax(child, depth + 1, alpha, beta, use_pruning))
        beta = min(beta, best)
        if use_pruning and beta <= alpha:
            break
    return best

plain = minimax(tree, 0, -999, 999, False)
plain_nodes = stats["plain"]
cut = minimax(tree, 0, -999, 999, True)
cut_nodes = stats["pruned"]

left = minimax(tree[0], 1, -999, 999, False)
right = minimax(tree[1], 1, -999, 999, False)

print("left branch value  : %d" % left)
print("right branch value : %d" % right)
print("root value, plain search  : %d" % plain)
print("root value, alpha-beta    : %d" % cut)
print("the maximiser chooses the left branch")
print()
```

Python (continued)

```python
print("internal nodes visited: plain %d, alpha-beta %d"
      % (plain_nodes, cut_nodes))
```

output

```
left branch value  : 3
right branch value : 1
root value, plain search  : 3
root value, alpha-beta    : 3
the maximiser chooses the left branch

internal nodes visited: plain 15, alpha-beta 11
```

Both methods return 3, and pruning visits 11 internal nodes instead of 15. The left branch earns its value because the second maximum picks 14 out of 4, 6, 14 and 5, while the first minimum there takes the smaller of the two maxima, 3 from `max(3, 12)` and 8 from `max(8, 2)`, so 3 wins.

Example 2: tic-tac-toe with minimax. X follows a fixed preference list, O plays perfect minimax with alpha-beta, and the node counts show what pruning saves.

Python

```python
"""Minimax and alpha-beta on tic-tac-toe.
X is a scripted player with a cell preference list; O plays perfectly."""

WIN_LINES = [(0, 1, 2), (3, 4, 5), (6, 7, 8),
             (0, 3, 6), (1, 4, 7), (2, 5, 8),
             (0, 4, 8), (2, 4, 6)]

def winner(board):
    for a, b, c in WIN_LINES:
        if board[a] != " " and board[a] == board[b] == board[c]:
            return board[a]
    return None

def full(board):
    return " " not in board

def free_cells(board):
    return [i for i, cell in enumerate(board) if cell == " "]

stats = [0]

def minimax(board, player, alpha, beta, use_pruning):
    stats[0] += 1
    win = winner(board)
    if win == "O":
        return 10 - board.count(" ")
    if win == "X":
        return board.count(" ") - 10
    if full(board):
        return 0
    if player == "O":
        best = -999
        for cell in free_cells(board):
            board[cell] = "O"
            best = max(best, minimax(board, "X", alpha, beta, use_pruning))
            board[cell] = " "
            alpha = max(alpha, best)
            if use_pruning and beta <= alpha:
                break
        return best
    best = 999
    for cell in free_cells(board):
        board[cell] = "X"
        best = min(best, minimax(board, "O", alpha, beta, use_pruning))
        board[cell] = " "
        beta = min(beta, best)
        if use_pruning and beta <= alpha:
            break
    return best

def best_move(board, player):
    best_score = None
    choice = None
    for cell in free_cells(board):
        board[cell] = player
        score = minimax(board, "X" if player == "O" else "O", -999, 999, True)
```

Python (continued)

```
        board[cell] = " "
        if best_score is None or score > best_score:
            best_score, choice = score, cell
    return choice, best_score

def show(board):
    return "\n".join(" " + " | ".join(board[r:r + 3]) + " " for r in range(0, 9, 3))

preference = [0, 4, 8, 2, 6, 1, 3, 5, 7]
board = [" "] * 9
turn, move_no = "X", 1

while winner(board) is None and not full(board):
    if turn == "X":
        cell = next(c for c in preference if board[c] == " ")
        board[cell] = "X"
        note = "X plays cell %d" % cell
    else:
        cell, score = best_move(board, "O")
        board[cell] = "O"
        note = "O plays cell %d (minimax score %d)" % (cell, score)
    print("move %d: %s" % (move_no, note))
    print(show(board))
    print()
    turn = "O" if turn == "X" else "X"
    move_no += 1

print("result:", winner(board) or "draw")

"""Node counts on a fixed mid-game position (X at 0 and 8, O at 4, O to move)."""
sample = ["X", " ", " ",
          " ", "O", " ",
          " ", " ", "X"]
stats[0] = 0
minimax(sample, "O", -999, 999, False)
plain_nodes = stats[0]
stats[0] = 0
minimax(sample, "O", -999, 999, True)
cut_nodes = stats[0]
print()
print("nodes visited on the sample position")
print("  plain minimax: %d" % plain_nodes)
print("  alpha-beta   : %d (%.1f%% as many)"
      % (cut_nodes, 100.0 * cut_nodes / plain_nodes))
```

output

```
move 1: X plays cell 0
 X |   |
   |   |
   |   |

move 2: O plays cell 4 (minimax score 0)
 X |   |
   | O |
   |   |

move 3: X plays cell 8
 X |   |
   | O |
   |   | X

move 4: O plays cell 1 (minimax score 0)
 X | O |
   | O |
   |   | X

move 5: X plays cell 2
 X | O | X
   | O |
   |   | X

move 6: O plays cell 5 (minimax score 9)
 X | O | X
   | O | O
   |   | X

move 7: X plays cell 6
 X | O | X
   | O | O
 X |   | X

move 8: O plays cell 3 (minimax score 9)
 X | O | X
 O | O | O
 X |   | X

result: O

nodes visited on the sample position
  plain minimax: 1053
  alpha-beta   : 332 (31.5% as many)
```

X takes the corners and the top row; O answers in the centre and then builds the column it is already holding. The printed scores are worth reading: the first two scores are 0 because the position is a draw with perfect play, and by move 6 the score is 9 because a win in two more moves is available. On the sample position, pruning examines 332 nodes instead of 1053, about a third, for the identical answer.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting the opponent moves too	the program finds its own best line and ignores replies	alternate maximiser and minimiser at every level

Mistake	Why it hurts	Best practice
Scoring only the final leaf	early wins are scored like late wins	subtract the number of moves left from a win
Updating alpha or beta with the wrong player's value	pruning cuts good branches	raise alpha in maximiser nodes, lower beta in minimiser nodes
Pruning before all children are tried in a shallow tree	the value changes silently	compare plain minimax against pruned minimax on small trees
Stopping at a fixed depth with no evaluation function	the leaf values become meaningless	write an evaluation that estimates who stands better
Assuming minimax handles chance	dice and shuffles need expected values	model chance nodes with averages over the possible deals

- When a game tree looks wrong, print the value of every child at the root and check the one your program picked against the others.
- Pruning is an optimisation, never a change of rules: if the two versions disagree, the pruning update is wrong.

4.5 Practice exercises

1. In Example 1, work out by hand the value of the node `((4, 6), (14, 5))` and the value of its parent.
2. Why does a win in fewer moves deserve a larger score than a win in more moves, in the program of Example 2?
3. Alpha-beta examined 332 of 1053 nodes on one position. Explain in two sentences why the value cannot change.
4. A game has dice. What kind of node must the tree contain, and how is its value computed?
5. The scripted player X in Example 2 loses. Name one opening move that would have drawn or won, and say why.

Solutions

1. The inner maxima give `max(4, 6) = 6` and `max(14, 5) = 14`, and the minimum above them takes `min(6, 14) = 6`. Its parent is also a minimum in the other branch of the left subtree, so it compares against the value 3 of the first maximum and contributes 3 to the left branch.
2. Because the score is `10 - empty cells`, a win recorded with fewer empty cells left is worth more; the program therefore prefers a win now over an equally certain win several moves later, which also avoids blunders in between.
3. Alpha-beta only skips a branch after the player to move already has a move at least as good as the value the branch could produce, so no skipped branch could have improved the answer; the skipped work was redundant.
4. A chance node, whose value is the average of the children's values weighted by the probabilities of the outcomes; children of the other players are still handled by minimax.

5. Playing the centre, cell 4, first. X's opening corner gives O the centre and the initiative; with the centre taken, the scripted corner play would have kept the game balanced for longer.

4.6 Mini test

1. Who maximises and who minimises in a minimax search? A) Both sides maximise B) The player to move at each level: one wants the largest value, the other the smallest C) The program maximises and never minimises D) The leaf nodes decide and the middle levels copy them
2. What value does a maximiser node return? A) The smallest child value B) The average of the child values C) The largest child value D) The value of the first child
3. Why was the score of O's sixth move in Example 2 equal to 9? A) Nine cells are empty B) A win is available, and `10 - 1` empty cell counts how soon it arrives C) Nine moves were played D) The board had nine marks
4. True or false: alpha-beta pruning can change the value at the root if the neighbour updates are correct.
5. What does an evaluation function do? A) Checks whether the game is over B) Estimates the worth of a position where the search stops C) Lists all legal moves D) Chooses the difficulty level
6. Which node type is missing from tic-tac-toe but present in backgammon? A) Terminal nodes B) Chance nodes C) Alternating decision nodes D) Leaf values

Answer key and explanations

1. B - the two sides have opposite objectives, so levels alternate between maximising and minimising.
2. C - a maximiser takes the best of its children, exactly as a minimiser takes the worst of its own.
3. B - the win score includes the remaining empty cells, so winning with one cell left is worth more than winning later.
4. False - pruning only removes branches that cannot change the result; the root value is identical.
5. B - when the tree is cut off early, the leaf must be valued by an estimate of the position rather than by a real result.
6. B - backgammon has dice, so its tree needs chance nodes with weighted averages.

Chapter 5 - Probability, odds and Bayes' rule

5.1 Summary

- Probability describes how often a statement is expected to hold in the long run, as a number between 0 and 1.
- Odds express the same uncertainty as a ratio, and converting between the two takes one division.
- Bayes' rule re-weighs hypotheses after new evidence, using how likely the evidence was under each hypothesis.
- The long form divides by the total probability of the evidence; the odds form multiplies by the likelihood ratio.
- A base rate can dominate a test, so an accurate test on a rare condition is wrong most of the time when it fires.
- Natural frequencies such as "out of ten thousand people" make the same arithmetic readable without formulas.
- Mutually exclusive outcomes add; independent events multiply.
- A simulation with a fixed seed shows the law of large numbers without any theory, since the running frequency settles.
- Every classifier in the rest of the volume rests on this chapter, and Naive Bayes is its most direct use.
- Writing the numbers down in the open is the cheapest guard against a probability mistake.

5.2 Concepts

The basics, written out

Idea	Rule	Example
Range	`0 <= P(A) <= 1`	a certain event is 1, an impossible one 0
Complement	`P(not A) = 1 - P(A)`	if rain is 0.2, no rain is 0.8
Mutually exclusive	`P(A or B) = P(A) + P(B)`	red or blue in one draw
Independent	`P(A and B) = P(A) * P(B)`	two fair coins
Conditional	`P(A and B) = P(A) * P(B given A)`	draw two cards without replacing
Total probability	`P(E) = sum over H of P(E given H) * P(H)`	a positive test in a mixed population

Bayes' rule in two clothes

output

```
long form                          odds form

          P(E | H) * P(H)              prior odds for H
P(H|E) = ---------------      x        likelihood ratio
                P(E)                posterior odds for H

          P(E given spam) * P(spam)    0.4 spam in 0.6 ham
P(spam | E) = -------------------      evidence 0.42
                    P(E)
```

Both compute the same number, and Example 1 prints them side by side. The odds form is usually faster: multiply two ratios instead of adding two products and dividing.

The base rate trap

Consider a test that is right 99 percent of the time on the sick and 95 percent right on the healthy.

Population	Base rate	Positive tests	Correct share of positives
Screening	0.1 percent sick	mostly healthy people	about 2 percent
Clinic	30 percent sick	mostly sick people	about 89 percent

The test did not change; the population did. Whenever a rare event is being tested, the false positives from the large healthy group can outnumber the true positives, and the ratio is decided by the base rate.

Counting instead of formulas

Step	Ask
1	how many people are in the imagined group, such as 10000
2	how many of them are in each hypothesis, such as 10 sick
3	how many of each group give the evidence, such as 9.9 and 499.5
4	divide the desired count by all counts with that evidence

5.3 Examples

Example 1: odds, probability and the Bayesian update in both forms, printing every number that enters the calculation.

Python

```python
"""Probability, odds and the Bayesian update, worked with numbers."""

def to_odds(p):
    return p / (1 - p)

def to_prob(odds_for):
    return odds_for / (1 + odds_for)

print("conversions")
print("  P(rain) = 0.20 -> odds 1 to %.0f" % (1 / to_odds(0.20)))
print("  P(win)  = 0.75 -> odds %.0f to 1" % to_odds(0.75))
print("  odds 3 to 1    -> P = %.2f" % to_prob(3))
print("  odds 1 to 4    -> P = %.2f" % to_prob(0.25))

"""One observation: the word "free" inside an e-mail."""
p_spam = 0.4
p_free_given_spam = 0.9
p_free_given_ham = 0.1

p_free = p_free_given_spam * p_spam + p_free_given_ham * (1 - p_spam)
posterior = p_free_given_spam * p_spam / p_free

odds_prior = to_odds(p_spam)
likelihood_ratio = p_free_given_spam / p_free_given_ham
odds_posterior = odds_prior * likelihood_ratio
posterior_odds = to_prob(odds_posterior)

print()
print("the long form of Bayes' rule")
print("  prior       P(spam)        = %.2f" % p_spam)
print("  likelihood  P(free | spam) = %.2f" % p_free_given_spam)
print("  likelihood  P(free | ham)  = %.2f" % p_free_given_ham)
print("  evidence    P(free)        = %.2f*%.2f + %.2f*%.2f = %.4f"
      % (p_free_given_spam, p_spam, p_free_given_ham, 1 - p_spam, p_free))
print("  posterior   P(spam | free) = %.4f / %.4f = %.4f"
      % (p_free_given_spam * p_spam, p_free, posterior))

print()
print("the odds form")
print("  prior odds       %.4f to 1" % odds_prior)
print("  likelihood ratio %.1f to 1" % likelihood_ratio)
print("  posterior odds   %.4f to 1" % odds_posterior)
print("  posterior P      %.4f" % posterior_odds)
```

output

```
conversions
  P(rain) = 0.20 -> odds 1 to 4
  P(win)  = 0.75 -> odds 3 to 1
  odds 3 to 1    -> P = 0.75
  odds 1 to 4    -> P = 0.20

the long form of Bayes' rule
  prior       P(spam)        = 0.40
  likelihood  P(free | spam) = 0.90
  likelihood  P(free | ham)  = 0.10
  evidence    P(free)        = 0.90*0.40 + 0.10*0.60 = 0.4200
  posterior   P(spam | free) = 0.3600 / 0.4200 = 0.8571

the odds form
  prior odds       0.6667 to 1
  likelihood ratio 9.0 to 1
  posterior odds   6.0000 to 1
  posterior P      0.8571
```

The prior is 0.4, that is two spams for every three hams. The evidence is nine times more likely in spam than in ham, so the odds multiply from 0.6667 to 6, which is a probability of about 0.857. Both routes agree, as they must.

Example 2: the same test in two populations, counted per ten thousand people.

Python

```python
"""A medical test with a low base rate: the same test, two populations."""

sensitivity = 0.99   # P(positive | sick)
specificity = 0.95   # P(negative | healthy)

def audit(prevalence, people=10000):
    sick = prevalence * people
    healthy = people - sick
    true_positive = sick * sensitivity
    false_negative = sick - true_positive
    true_negative = healthy * specificity
    false_positive = healthy - true_negative
    ppv = true_positive / (true_positive + false_positive)
    print("base rate %.1f%%, %d people tested"
          % (prevalence * 100, people))
    print("  actually sick     %8.1f" % sick)
    print("  actually healthy  %8.1f" % healthy)
    print("  true positives    %8.1f" % true_positive)
    print("  false negatives   %8.1f" % false_negative)
    print("  false positives   %8.1f" % false_positive)
    print("  true negatives    %8.1f" % true_negative)
    print("  P(sick | positive) = %.1f / %.1f = %.4f"
          % (true_positive, true_positive + false_positive, ppv))
    print()

audit(0.001)
audit(0.30)
print("the test itself never changed; the population did")
```

output

```
base rate 0.1%, 10000 people tested
  actually sick         10.0
  actually healthy    9990.0
  true positives         9.9
  false negatives        0.1
  false positives      499.5
  true negatives      9490.5
  P(sick | positive) = 9.9 / 509.4 = 0.0194

base rate 30.0%, 10000 people tested
  actually sick       3000.0
  actually healthy    7000.0
  true positives      2970.0
  false negatives       30.0
  false positives      350.0
  true negatives      6650.0
  P(sick | positive) = 2970.0 / 3320.0 = 0.8946

the test itself never changed; the population did
```

In the screening group only about 2 percent of positive results are correct, because 499.5 healthy people produce false alarms against 9.9 true ones. In the clinic group the same test is right about 89 percent of the time. The numbers are not a pathology of this example; they are what a good test does when the condition is rare.

Example 3: a simulation that shows how running frequency settles towards theory, using a fixed seed so the numbers are reproducible.

Python

```python
"""Simulation with a fixed seed: empirical frequency versus theory."""

import random

random.seed(42)
flips = [random.choice("HT") for _ in range(2000)]

print("heads in 2000 flips: %d (%.4f)"
      % (flips.count("H"), flips.count("H") / 2000))
print("theory says 1000 (0.5000)")
print()
for n in (10, 100, 1000, 2000):
    print("after %4d flips: %.3f" % (n, flips[:n].count("H") / n))

after_heads = [flips[i + 1] for i in range(len(flips) - 1) if flips[i] == "H"]
after_tails = [flips[i + 1] for i in range(len(flips) - 1) if flips[i] == "T"]
print()
print("P(next H | previous H) = %.4f"
      % (after_heads.count("H") / len(after_heads)))
print("P(next H | previous T) = %.4f"
      % (after_tails.count("H") / len(after_tails)))

longest = run = 0
for f in flips:
    run = run + 1 if f == "H" else 0
    longest = max(longest, run)
print()
print("longest run of heads: %d" % longest)
```

output

```
heads in 2000 flips: 967 (0.4835)
theory says 1000 (0.5000)

after   10 flips: 0.800
after  100 flips: 0.560
after 1000 flips: 0.486
after 2000 flips: 0.483

P(next H | previous H) = 0.4860
P(next H | previous T) = 0.4806

longest run of heads: 10
```

After ten flips the frequency is 0.8 and little can be concluded; after two thousand it is 0.4835, close to the theoretical 0.5. The two conditional frequencies are close to each other, which is what independence means: the coin does not remember. A run of ten heads in two thousand flips is not evidence of a rigged coin, and this simulation is one such case.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading a high accuracy as a high probability	accuracy mixes base rates together	always state the population as well
Confusing `P(A given B)` with `P(B given A)`	the medical trap in Example 2	label every probability with what is known and what is asked
Adding probabilities that are not exclusive	the sum can exceed 1	check whether both events can happen at once
Multiplying probabilities of dependent events	the product is too small	use the conditional form
Trusting a small simulation	ten flips prove nothing	increase the count and print the running frequency
Hiding the arithmetic	mistakes survive review	print each term, as the examples here do

- When a probability problem feels hard, count people instead of writing formulas.
- Quote a probability together with the group it refers to; a bare number hides the base rate.

5.5 Practice exercises

1. Convert a probability of 0.6 to odds, and odds of 5 to 2 to a probability.
2. A spam filter sees the word "prize". Prior odds of spam are 1 to 1, and "prize" is 4 times more likely in spam than in ham. What are the posterior odds and probability?
3. Recompute Example 2 with a base rate of 1 percent by hand, then check with the program by calling `audit(0.01)`.
4. Two fair dice are rolled. What is the probability that the sum is 7, and what is the probability that at least one die shows 6?

5. A test is 90 percent accurate on the sick and 90 percent accurate on the healthy, with a base rate of 50 percent. Explain why the base rate trap is absent here.

Solutions

1. Odds are `0.6 / 0.4 = 1.5`, that is 3 to 2. Odds of 5 to 2 mean `5 / (5 + 2) = 0.7143`.
2. Posterior odds are 4 to 1, so the probability is `4 / 5 = 0.8`.
3. Out of 10000 people there are 100 sick and 9900 healthy. True positives are 99, false positives 495, so `99 / (99 + 495) = 0.1667`, about 17 percent. The program prints the same figure when called with 0.01.
4. Six of the thirty-six ordered pairs sum to 7, so the probability is `6/36 = 0.1667`. At least one six occurs in eleven of the thirty-six pairs, so the probability is `11/36 = 0.3056`.
5. When the base rate is 50 percent, the sick and healthy groups are the same size, so the false positives from the healthy group and the true positives from the sick group are comparable; the base rate no longer dominates the arithmetic.

5.6 Mini test

1. What are the odds of probability 0.8? A) 1 to 8 B) 4 to 1 C) 8 to 1 D) 1 to 4
2. Which quantity multiplies the prior odds in the odds form of Bayes' rule? A) The posterior probability B) The evidence probability C) The likelihood ratio D) The base rate
3. A test has 99 percent sensitivity and 95 percent specificity. The base rate is 0.1 percent. What share of positive results is correct? A) 99 percent B) About 50 percent C) About 2 percent D) 95 percent
4. True or false: an accurate test always becomes more correct when the base rate falls.
5. Which rule computes `P(A and B)` for independent events? A) `P(A) + P(B)` B) `P(A) * P(B)` C) `P(A) / P(B)` D) `1 - P(A) * P(B)`
6. Why did Example 3 use a fixed seed? A) To make the coins heavier B) To make the printed frequencies reproducible on every machine C) To remove the need for the `random` module D) To make the flips dependent on each other

Answer key and explanations

1. B - `0.8 / 0.2 = 4`, so the odds are 4 to 1.
2. C - the odds form multiplies prior odds by the likelihood ratio, which is `P(evidence | hypothesis)` divided by `P(evidence | alternative)`.
3. C - in a rare condition, false positives from the large healthy group dominate the positive results, about 499 against 10.
4. False - a falling base rate makes positive results less reliable, not more, because the healthy group grows relative to the sick group.
5. B - independence is exactly what allows the product rule.
6. B - a seeded generator produces the same sequence everywhere, so the printed frequencies can be checked line by line.

Chapter 6 - Naive Bayes classification in practice

6.1 Summary

- Naive Bayes compares the posterior probability of each class for the words in a message and picks the larger one.
- The prior comes from how often a class appears in the training data, here the count of messages of each kind.
- The likelihood of a word comes from how often that word appears inside the messages of one class.
- Multiplying many small probabilities underflows to zero, so the sums are taken in logarithms and the larger total wins.
- Laplace smoothing adds one to every word count so that a word never seen before does not make the whole message impossible.
- The "naive" part is the assumption that words are independent given the class, which is false in real text but works well in practice.
- A tiny hand-made data set is enough to build the model and to read every number that decides a verdict.
- Leave-one-out evaluation retrains without one message at a time, giving an honest estimate when data is scarce.
- The model fails quietly when a single word carries the decision or when a class is missing from training.
- Inspecting the one mistake a small model makes teaches more than the accuracy figure.

6.2 Concepts

What the model needs

Ingredient	Where it comes from	In the example
Prior `P(class)`	message counts per class	6 ham and 6 spam, so both are 0.5
Likelihood `P(word given class)`	word counts inside that class	"free" appears 4 times in spam
Vocabulary size `V`	distinct words seen in training	53 words
Smoothing `alpha`	a choice, usually 1	adds 1 to every count, `V` to every total

The formula behind the code

output

```
P(class | words) is proportional to

    P(class) * product over words of P(word | class)

log form, used by the program:

log P(ham  | message) = log P(ham)  + sum of log P(word | ham )
log P(spam | message) = log P(spam) + sum of log P(word | spam)

the class with the larger log total wins, and the logs only matter
relatively, so the normalising constant is not needed
```

Smoothing, in numbers

A word that never appears in a class would get likelihood 0 and would wipe out the whole message. Laplace smoothing instead uses `(count + 1) / (total + V)`, so an unseen word keeps a small but non-zero probability.

Word	Count in ham	Count in spam	P(word given ham)	P(word given spam)
free	0	4	1 / 35 = 0.0286	5 / 35 = 0.1429
meeting	1	0	2 / 35 = 0.0571	1 / 35 = 0.0286
cash	0	1	1 / 35 = 0.0286	2 / 35 = 0.0571

The ham total is 34 words and the spam total 34 words, with `V = 53`, so every denominator is `34 + 53 = 87` before the counts are added. The table above shows the smoothed probabilities, and the program prints the logarithms of the same quantities.

The pipeline

output

```
message
   |
   v
split into words          ["free", "cash", "prize", "now"]
   |
   +--> look up the word count in each class
   |
   v
start from the log prior, add one log likelihood per word
   |
   v
two log totals ---------> the larger one names the verdict
```

6.3 Examples

Example 1: a spam filter written from scratch on twelve hand-made messages, with the likelihood table and the full calculation for one verdict.

Python

```python
"""Naive Bayes for spam on a tiny hand-made e-mail set."""

import math

ham = [
    "meeting moved to three pm",
    "please send the report today",
    "lunch on friday sounds good",
    "the project deadline is next week",
    "can we talk tomorrow morning",
    "thanks for the quick reply",
]

spam = [
    "free money claim your prize now",
    "urgent you won a free phone",
    "claim your free lottery ticket",
    "limited offer buy now and save",
    "you are selected for a free gift",
    "click now to win cash prize",
]

def tokens(text):
    return text.split()

def build(messages):
    counts, total = {}, 0
    for message in messages:
        for token in tokens(message):
            counts[token] = counts.get(token, 0) + 1
            total += 1
    return {"counts": counts, "total": total, "n": len(messages),
            "vocab": set(counts)}

HAM = build(ham)
SPAM = build(spam)
VOCAB = HAM["vocab"] | SPAM["vocab"]
V = len(VOCAB)
N = HAM["n"] + SPAM["n"]

log_prior_ham = math.log(HAM["n"] / N)
log_prior_spam = math.log(SPAM["n"] / N)

def log_like(token, model):
    return math.log((model["counts"].get(token, 0) + 1)
                    / (model["total"] + V))

def scores(text):
    s_h, s_s = log_prior_ham, log_prior_spam
    for token in tokens(text):
        s_h += log_like(token, HAM)
        s_s += log_like(token, SPAM)
    return s_h, s_s

def classify(text):
    s_h, s_s = scores(text)
```

Python (continued)

```
    return ("spam" if s_s > s_h else "ham"), s_h, s_s

print("training messages: %d ham, %d spam" % (HAM["n"], SPAM["n"]))
print("vocabulary: %d distinct words" % V)
print("log P(ham) = %.4f, log P(spam) = %.4f"
      % (log_prior_ham, log_prior_spam))
print()
print("word likelihoods with Laplace smoothing (alpha = 1)")
print("%-10s %10s %10s" % ("word", "P(w|ham)", "P(w|spam)"))
for word in ("free", "prize", "meeting", "report", "now", "lunch"):
    print("%-10s %10.4f %10.4f"
          % (word, math.exp(log_like(word, HAM)),
             math.exp(log_like(word, SPAM))))

print()
print("worked example: 'free cash prize now'")
for token in tokens("free cash prize now"):
    print("  + %-8s log P(token|ham) = %8.4f   log P(token|spam) = %8.4f"
          % (token, log_like(token, HAM), log_like(token, SPAM)))
verdict, s_h, s_s = classify("free cash prize now")
print("  total log P(message|ham)  = %8.4f" % s_h)
print("  total log P(message|spam) = %8.4f" % s_s)
print("  verdict: %s" % verdict)

print()
print("three test messages")
for message in ("free cash prize now", "meeting moved to friday",
                "please claim your free report"):
    verdict, s_h, s_s = classify(message)
    print("  %-30s -> %-4s (ham %.3f, spam %.3f)"
          % (message, verdict, s_h, s_s))
```

output

```
training messages: 6 ham, 6 spam
vocabulary: 53 distinct words
log P(ham) = -0.6931, log P(spam) = -0.6931

word likelihoods with Laplace smoothing (alpha = 1)
word         P(w|ham)  P(w|spam)
free           0.0119     0.0562
prize          0.0119     0.0337
meeting        0.0238     0.0112
report         0.0238     0.0112
now            0.0119     0.0449
lunch          0.0238     0.0112

worked example: 'free cash prize now'
  + free     log P(token|ham) =  -4.4308   log P(token|spam) =  -2.8792
  + cash     log P(token|ham) =  -4.4308   log P(token|spam) =  -3.7955
  + prize    log P(token|ham) =  -4.4308   log P(token|spam) =  -3.3900
  + now      log P(token|ham) =  -4.4308   log P(token|spam) =  -3.1023
  total log P(message|ham)  = -18.4164
  total log P(message|spam) = -13.8602
  verdict: spam

three test messages
  free cash prize now            -> spam (ham -18.416, spam -13.860)
  meeting moved to friday        -> ham  (ham -15.644, spam -17.955)
  please claim your free report  -> spam (ham -21.461, spam -19.330)
```

Every printed number can be checked: "free" appears in 4 of the 6 spam messages and in none of the ham messages, so with smoothing its ham probability is `1 / 87 = 0.0115` and its spam probability `5 / 87 = 0.0575`, close to the printed values. The message "meeting moved to friday" wins for ham because "meeting" only occurs in ham. The third message is genuinely ambiguous: "claim" and "free" point to spam, while "please" and "report" point to ham, and spam wins by a small margin.

Example 2: leave-one-out evaluation, which trains on eleven messages and tests on the one left out, twelve times over.

Python

```python
"""Leave-one-out evaluation of the same tiny Naive Bayes model."""

import math

MESSAGES = [
    ("meeting moved to three pm", "ham"),
    ("please send the report today", "ham"),
    ("lunch on friday sounds good", "ham"),
    ("the project deadline is next week", "ham"),
    ("can we talk tomorrow morning", "ham"),
    ("thanks for the quick reply", "ham"),
    ("free money claim your prize now", "spam"),
    ("urgent you won a free phone", "spam"),
    ("claim your free lottery ticket", "spam"),
    ("limited offer buy now and save", "spam"),
    ("you are selected for a free gift", "spam"),
    ("click now to win cash prize", "spam"),
]

def build(messages):
    counts, total = {}, 0
    for message in messages:
        for token in message.split():
            counts[token] = counts.get(token, 0) + 1
            total += 1
    return {"counts": counts, "total": total, "vocab": set(counts)}

def toks(message):
    return message.split()

def train(messages):
    models = {}
    for label in ("ham", "spam"):
        subset = [m for m, l in messages if l == label]
        models[label] = build(subset)
        models[label]["n"] = len(subset)
    n_all = len(messages)
    vocab = models["ham"]["vocab"] | models["spam"]["vocab"]
    return models, vocab, n_all

def classify(message, models, vocab, n_all):
    V, best, best_score = len(vocab), None, None
    for label in ("ham", "spam"):
        model = models[label]
        score = math.log(model["n"] / n_all)
        for token in toks(message):
            score += math.log((model["counts"].get(token, 0) + 1)
                              / (model["total"] + V))
        if best_score is None or score > best_score:
            best, best_score = label, score
    return best

correct = 0
for i, (message, label) in enumerate(MESSAGES):
    rest = MESSAGES[:i] + MESSAGES[i + 1:]
    models, vocab, n_all = train(rest)
```

Python (continued)

```
    guess = classify(message, models, vocab, n_all)
    ok = guess == label
    correct += ok
    print("%-32s true %-4s predicted %-4s %s"
          % (message, label, guess, "ok" if ok else "WRONG"))

print()
print("leave-one-out accuracy: %d/%d = %.1f%%"
      % (correct, len(MESSAGES), 100.0 * correct / len(MESSAGES)))
```

output

```
meeting moved to three pm        true ham  predicted spam WRONG
please send the report today     true ham  predicted ham  ok
lunch on friday sounds good      true ham  predicted ham  ok
the project deadline is next week true ham  predicted ham  ok
can we talk tomorrow morning     true ham  predicted ham  ok
thanks for the quick reply       true ham  predicted ham  ok
free money claim your prize now  true spam predicted spam ok
urgent you won a free phone      true spam predicted spam ok
claim your free lottery ticket   true spam predicted spam ok
limited offer buy now and save   true spam predicted spam ok
you are selected for a free gift true spam predicted spam ok
click now to win cash prize      true spam predicted spam ok

leave-one-out accuracy: 11/12 = 91.7%
```

The single mistake is instructive. When "meeting moved to three pm" is held out, the word "meeting" is still in training, but the shifted priors and the short length of the message pull the total towards spam; the word "to" and the overall short length are the cause. A dataset of twelve messages cannot support a stronger claim, and that is the honest reading of a 91.7 percent score.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Multiplying raw probabilities	long messages underflow to exactly zero	work in logarithms and compare sums
Skipping smoothing	one unseen word zeroes a whole message	add one to each count and `V` to each total
Testing on the training rows	the score measures memory, not skill	hold rows out, or use leave-one-out
Trusting one confident class	a single word can dominate a short message	inspect the words that decided the verdict
Assuming word independence is true	it is false, and it fails when word pairs matter	keep the assumption, but test on longer real text
Ignoring the class balance	a missing class silently becomes the default	print the priors and check the counts

- Print the log totals for both classes before believing a verdict; the two numbers and their difference tell the whole story.

- Logs also turn multiplication into addition, which is why the model stays readable even with hundreds of words per message.

6.5 Practice exercises

1. In Example 1, "cash" never occurs in the ham messages. Compute its smoothed likelihood for ham and for spam by hand, given 34 ham words, 34 spam words and a vocabulary of 53.
2. Why does the program compare the two log totals instead of computing the probability of each class exactly?
3. The message "please claim your free report" is classified as spam. Which words push towards spam, and which push towards ham?
4. A training set contains 90 ham messages and 10 spam messages. What prior does the model learn, and why can that be a problem for recall?
5. Suggest one experiment, using the twelve messages, that would show whether smoothing is helping or hurting this classifier.

Solutions

1. For ham: `(0 + 1) / (34 + 53) = 1/87 = 0.0115`. For spam: `(1 + 1) / 87 = 2/87 = 0.0230`. The spam likelihood is exactly twice the ham likelihood, because smoothing makes the difference come from the count alone.
2. Because the true posterior needs the total probability of the message, which is the same number for both classes; whoever has the larger unnormalised total also has the larger posterior, so the division can be skipped.
3. "claim" and "free" push towards spam, "please" and "report" push towards ham, and spam wins by about 2.1 in log units.
4. The model learns `log(0.9)` for ham against `log(0.1)` for spam, a handicap of about 2.2 log units for every spam message. Since spam is the interesting class, recall for spam suffers unless the evidence is strong.
5. Run the leave-one-out evaluation twice, once with `alpha = 1` and once with `alpha = 0.1`, and compare the mistake each version makes; the difference shows how much unseen words matter.

6.6 Mini test

1. Which two quantities are added in log space to score a message? A) The word counts and the class count B) The log prior and the log likelihoods of the words C) The vocabulary size and the message length D) The accuracy and the error rate
2. Why is Laplace smoothing applied? A) To make the priors equal B) So that a word missing from a class does not give probability zero C) To shorten the vocabulary D) To replace logarithms with products
3. What does the "naive" in Naive Bayes refer to? A) That the model is a beginner's tool B) That words are treated as independent given the class C) That the training set is small D)

That the verdict is random

4. True or false: leave-one-out evaluation trains on all the messages including the one being tested.
5. The third test message is close to the boundary. What is the honest description of that verdict? A) The model is broken B) The evidence is weak and either call is defensible C) The model is 100 percent sure D) The word "please" decided it alone
6. If ham messages outnumber spam messages 9 to 1, what happens to the prior for spam? A) It becomes zero B) It becomes 0.1, a handicap in log units against spam C) It stays 0.5 because Bayes normalises D) It depends on the words

Answer key and explanations

1. B - the log prior starts the total and one log likelihood is added per word.
2. B - without smoothing a single unseen word would zero the whole message probability.
3. B - the independence assumption is the definition of the model, and it is knowingly wrong for real text.
4. False - the held-out message is removed before training, which is what makes the estimate honest.
5. B - the two totals differ by about 2 log units out of roughly 20, so the verdict is close to the boundary.
6. B - the prior is `10 / 100 = 0.1`, which costs spam about 2.2 log units before any evidence.

Chapter 7 - The types of machine learning

7.1 Summary

- Supervised learning fits a mapping from inputs to known outputs using labelled examples.
- Unsupervised learning looks for structure in data that carries no labels, for example clusters or rare points.
- Reinforcement learning learns by acting, receiving rewards and updating what it expects to gain.
- Semi-supervised, self-supervised and transfer learning are combinations that appear as soon as real projects start.
- A baseline that repeats the most common label is the cheapest way to know whether a model is worth anything.
- Splitting rows into training and test parts is what makes an accuracy number mean anything at all.
- A clustering method such as k-means needs a distance, a number of clusters and starting points, and the last one is arbitrary.
- The reinforcement learning loop of state, action, reward and next state is captured by one small update rule.

- The same table of rows can be attacked by several learning types, and they answer different questions about it.
- Knowing which type you are using decides what data you must go and collect.

7.2 Concepts

The three main types

Type	Input data	What comes out	Everyday example
Supervised	rows with input features and a target label	a rule mapping features to label	spam or not, price tomorrow
Unsupervised	rows with features only	groups, distances, anomalies	customer segments, fraud outliers
Reinforcement	states, actions and rewards	a policy choosing actions	game playing, robot control

Labelled, unlabelled and rewarded

output

```
supervised                unsupervised              reinforcement
----------                ------------             -------------

features + label  ------>  model ------>  label      state ----.
      |                                              |         |
      v                                              v         v
learn a mapping           look for groups        action -> reward
from known answers        without answers        repeat, keep what pays
```

The words used when data is split

Term	Meaning	In the example
Feature	an input column the model may read	hours studied, hours slept
Label	the answer the model predicts	pass or fail
Training set	rows used to fit the model	8 rows
Test set	rows kept aside to measure	4 rows
Baseline	a trivial rule to beat	always predict the most common label
Split	the rule that decides which rows go where	every third row is held out

What k-means needs

Ingredient	Choice	Consequence
Distance	Euclidean here	decides what "close" means

Ingredient	Choice	Consequence
Number of clusters k	given, not learned	too few merges groups, too many splits them
Starting centres	two sample points	the answer depends on the start
Stopping	assignments stop changing	the result is a local, not a global, optimum

7.3 Examples

Example 1: a hand-written split and a baseline, which every fancier model must beat before it is interesting.

Python

```
"""Features, labels, a hand-written split and a baseline to beat."""

students = [
    (4, 7, "pass"),
    (6, 6, "pass"),
    (2, 5, "fail"),
    (8, 8, "pass"),
    (1, 4, "fail"),
    (5, 6, "pass"),
    (3, 5, "fail"),
    (7, 9, "pass"),
    (2, 6, "fail"),
    (9, 5, "pass"),
    (0, 3, "fail"),
    (6, 8, "pass"),
]

train = [row for i, row in enumerate(students) if i % 3 != 0]
test = [row for i, row in enumerate(students) if i % 3 == 0]

print("training rows: %d, test rows: %d" % (len(train), len(test)))
print("%-8s %-6s %-10s %s" % ("hours", "sleep", "label", "part"))
for hours, sleep, label in students:
    part = "train" if (hours, sleep, label) in train else "test"
    print("%-8d %-6d %-10s %s" % (hours, sleep, label, part))

labels = [row[2] for row in train]
majority = max(sorted(set(labels)), key=labels.count)
correct = sum(1 for row in test if row[2] == majority)
print()
print("most common label in train: %s (%d of %d)"
      % (majority, labels.count(majority), len(labels)))
print("baseline rule: always predict %r" % majority)
print("baseline accuracy on test: %d/%d = %.2f"
      % (correct, len(test), correct / len(test)))
```

output

```
training rows: 8, test rows: 4
hours    sleep  label      part
4        7      pass       test
6        6      pass       train
2        5      fail       train
8        8      pass       test
1        4      fail       train
5        6      pass       train
3        5      fail       test
7        9      pass       train
2        6      fail       train
9        5      pass       test
0        3      fail       train
6        8      pass       train

most common label in train: fail (4 of 8)
baseline rule: always predict 'fail'
baseline accuracy on test: 1/4 = 0.25
```

The training set holds four pass and four fail rows, so the majority class is exactly tied. The script breaks the tie by taking the first label in alphabetical order, which is an arbitrary choice: on these four test rows a model that always says "pass" would score 0.75 and one that always says "fail" scores 0.25. The lesson is that a baseline has to be stated explicitly, because on a small test set the choice of trivial rule can move the score further than a real model does. The split line uses `sorted(set(...))` for the same reason: a tie in `max` over a plain set would depend on the interpreter's hash seed and would not reproduce.

Example 2: k-means, an unsupervised method, on six unlabelled points. The starting centres are drawn with a fixed seed so the result repeats.

Python

```python
"""k-means: an unsupervised method on six unlabelled points."""

import random

points = [(1.0, 1.0), (1.5, 2.0), (3.0, 4.0),
          (5.0, 7.0), (3.5, 5.0), (4.5, 5.0)]
random.seed(42)

def distance(a, b):
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

centers = random.sample(points, 2)
print("starting centers: %s" % (centers,))

for round_no in range(1, 5):
    groups = [[] for _ in centers]
    for point in points:
        best = min(range(len(centers)), key=lambda i: distance(point, centers[i]))
        groups[best].append(point)
    print()
    print("round %d" % round_no)
    for i, (center, group) in enumerate(zip(centers, groups)):
        print("  center %s, members: %s" % (center, group))
    centers = [(sum(p[0] for p in g) / len(g), sum(p[1] for p in g) / len(g))
               for g in groups if g]
    print("  new centers: %s" % (centers,))

inertia = 0.0
for point in points:
    inertia += min(distance(point, c) ** 2 for c in centers)
print()
print("final centers: %s" % (centers,))
print("within-cluster sum of squares: %.3f" % inertia)
```

output

```
starting centers: [(4.5, 5.0), (1.0, 1.0)]

round 1
  center (4.5, 5.0), members: [(3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0)]
  center (1.0, 1.0), members: [(1.0, 1.0), (1.5, 2.0)]
  new centers: [(4.0, 5.25), (1.25, 1.5)]

round 2
  center (4.0, 5.25), members: [(3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0)]
  center (1.25, 1.5), members: [(1.0, 1.0), (1.5, 2.0)]
  new centers: [(4.0, 5.25), (1.25, 1.5)]

round 3
  center (4.0, 5.25), members: [(3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0)]
  center (1.25, 1.5), members: [(1.0, 1.0), (1.5, 2.0)]
  new centers: [(4.0, 5.25), (1.25, 1.5)]

round 4
  center (4.0, 5.25), members: [(3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0)]
  center (1.25, 1.5), members: [(1.0, 1.0), (1.5, 2.0)]
  new centers: [(4.0, 5.25), (1.25, 1.5)]

final centers: [(4.0, 5.25), (1.25, 1.5)]
within-cluster sum of squares: 7.875
```

The seed picks `(4.5, 5.0)` and `(1.0, 1.0)` as the first centres, and after round 1 the centres stop moving. The within-cluster sum of squares, the sum of each point's squared distance to its own centre, is the quantity the algorithm lowers, and it is what you would compare between different values of `k`.

Example 3: one Q-learning update by hand, then a loop that teaches a corridor.

Python

```python
"""Reinforcement learning: one Q update by hand, then a tiny Q-learning loop."""

"""Part A - a single update of the rule Q(s,a) <- Q(s,a) + alpha * (target - Q)"""
q_old = 0.0
alpha, gamma, reward = 0.5, 0.9, 0.0
best_next_q = 0.5

target = reward + gamma * best_next_q
q_new = q_old + alpha * (target - q_old)

print("part A: one update")
print("  Q(s, a) before : %.4f" % q_old)
print("  target         : reward %.1f + gamma %.1f * max Q(s') %.1f = %.4f"
      % (reward, gamma, best_next_q, target))
print("  new Q(s, a)    : %.4f + %.1f * (%.4f - %.4f) = %.4f"
      % (q_old, alpha, target, q_old, q_new))

"""Part B - a corridor of five cells, the goal sits at the far right."""
import random

STATES, GOAL, ACTIONS = 5, 4, (0, 1)          # 0 = left, 1 = right
Q = [[0.0, 0.0] for _ in range(STATES)]
random.seed(42)

solved = 0
episode_steps = []
for episode in range(300):
    state = 0
    steps = 0
    for _ in range(30):
        if random.random() < 0.2:
            action = random.choice(ACTIONS)     # explore
        else:
            action = 1 if Q[state][1] >= Q[state][0] else 0   # exploit
        nxt = min(max(state + (1 if action else -1), 0), STATES - 1)
        reward = 1.0 if nxt == GOAL else 0.0
        old = Q[state][action]
        Q[state][action] = old + 0.5 * (
            reward + 0.9 * max(Q[nxt]) - old)
        state = nxt
        steps += 1
        if nxt == GOAL:
            solved += 1
            break
    episode_steps.append(steps)

print()
print("part B: 300 episodes, epsilon 0.2, alpha 0.5, gamma 0.9")
print("episodes that reached the goal: %d/300" % solved)
print("steps in the first episode: %d, in the last: %d"
      % (episode_steps[0], episode_steps[-1]))
print()
print("learned Q table (left, right)")
for state, row in enumerate(Q):
    print("  cell %d: %7.3f %7.3f" % (state, row[0], row[1]))
```

output

```
part A: one update
  Q(s, a) before : 0.0000
  target         : reward 0.0 + gamma 0.9 * max Q(s') 0.5 = 0.4500
  new Q(s, a)    : 0.0000 + 0.5 * (0.4500 - 0.0000) = 0.2250

part B: 300 episodes, epsilon 0.2, alpha 0.5, gamma 0.9
episodes that reached the goal: 300/300
steps in the first episode: 6, in the last: 4

learned Q table (left, right)
  cell 0:   0.656   0.729
  cell 1:   0.656   0.810
  cell 2:   0.729   0.900
  cell 3:   0.810   1.000
  cell 4:   0.000   0.000
```

Every state except the goal prefers moving right, and the table is larger further right because the reward is one step closer. Nothing told the agent the shape of the corridor; it discovered it by walking.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Skipping the baseline	a 75 percent score looks like skill until the baseline also scores 75 percent	always print the trivial rule's score
Leaking test rows into training	the score is optimistic and meaningless	split first, then fit, and never touch the test rows
Treating clusters as truths	k-means finds groups, not necessarily real ones	compare several k values and look at what changed
Assuming a fixed number of clusters is known	the answer can be arbitrary	try several k and describe what the groups mean
Forgetting exploration	an agent that never tries anything learns a bad policy	keep an exploration rate and lower it slowly
Choosing a distance without thought	the clusters inherit the distance's opinion	say what the distance means before clustering

- Before any model, write down what the trivial baselines would score.
- Clusters are a description of your data, not a fact about the world, until a person can name what each group is.

7.5 Practice exercises

1. Classify each task as supervised, unsupervised or reinforcement learning: grouping news articles by topic; predicting tomorrow's temperature from a week of readings; teaching a robot arm to stack blocks; flagging transactions that look unusual; learning to steer a car inside a simulator.
2. Why must the baseline be computed on the test rows and not on the training rows?

3. In Example 2, would a different random seed possibly give different final centres? Explain what would and would not change.
4. The Q table shows 0 for both actions in cell 4. Why is that acceptable?
5. Suggest one extra feature for the students table that would probably make a supervised model better, and say which type of learning would need no labels at all for the same table.

Solutions

1. Grouping articles: unsupervised. Predicting temperature: supervised. Stacking blocks with a robot: reinforcement. Flagging unusual transactions: usually unsupervised, or supervised if past fraud labels exist. Steering inside a simulator: reinforcement.
2. Because the baseline is a claim about performance on unseen rows; measuring it on training rows would let it look good for the same reason a memorising model looks good.
3. Yes, other centres are possible at the start, so the first assignment can differ. Nothing else changes: the distance, the data and the objective are the same, and any fixed point of the same iterations is still a local optimum.
4. Cell 4 is the goal, where the episode ends, so no action is ever evaluated there; the zeros mean "never used", not "worthless".
5. Attendance at practical sessions, or the number of previous attempts. A clustering method needs no labels, so it could group the students by behaviour without knowing who passed.

7.6 Mini test

1. Which learning type needs labelled examples? A) Unsupervised learning B) Supervised learning C) Reinforcement learning D) Clustering
2. What does k-means produce? A) A label for new rows B) A grouping of the rows and a centre for each group C) A probability for each row D) A ranking of the features
3. What is a baseline? A) The best model available B) A trivial rule that a real model must beat C) The average of the labels D) The first training row
4. True or false: holding out every third row as test data guarantees the test rows differ from the training rows.
5. In the Q update, what does `gamma` discount? A) The exploration rate B) Future reward relative to immediate reward C) The learning rate D) The number of episodes
6. Which statement about the corridor agent is correct? A) It was told the goal was to the right B) It learned the values from rewards alone C) It needed a map of the corridor D) It could not learn with exploration

Answer key and explanations

1. B - supervised learning is defined by training on input and target pairs.
2. B - k-means outputs group membership and the centre of each group.
3. B - a baseline is the trivial comparison that keeps model scores honest.
4. False - every third row is chosen by position, so a regular pattern in the rows could make the test rows systematically different; random or representative splits are safer.

5. B - `gamma` below 1 makes a reward one step later worth less than the same reward now.
6. B - the agent knew only its state, its two actions and the reward at the goal, and the Q table emerged from those.

Chapter 8 - Nearest neighbours and regression

8.1 Summary

- k-nearest neighbours classifies a row by the labels of the training rows closest to it.
- Distance defines closeness, and choosing a distance is a modelling decision, not a detail.
- A small k follows noise, a large k smooths away small classes, and k = 1 simply memorises the training rows.
- Weighting each neighbour by one over its distance gives ties a sensible resolution.
- Simple linear regression fits a line by minimising the sum of squared errors.
- Gradient descent follows the slope of the loss and arrives at the same line as the closed-form formula.
- The two fitted numbers are directly readable as the slope and the intercept of the line.
- Feature scaling changes what "near" means whenever columns live on different scales.
- Both methods keep the training data itself, paying the cost of looking at every row when a prediction is made.
- Because both can be checked by hand, they are good first models and good debuggers for everything that follows.

8.2 Concepts

What k-NN needs

Ingredient	Choice	Effect
Number of neighbours `k`	odd, to avoid ties	small k is noisy, large k is smooth
Distance	Euclidean for these examples	decides what counts as close
Vote	one vote per neighbour	simple, and each row counts equally
Tie-break	weight by `1 / distance`	a nearer neighbour decides a tied vote
Storage	the whole training set	no training time, all the cost at prediction

Reading a distance table

For the query point in Example 1, the ten training rows rank like this, and the rank decides the answer:

output

```
distance 1.503  (5.5, 2.5)  blue   <-- k = 1 stops here
distance 1.887  (3.0, 1.0)  red
distance 2.088  (2.0, 2.0)  red    <-- k = 3 reaches here: blue 1, red 2
distance 2.193  (2.0, 3.5)  red
distance 2.283  (6.0, 1.5)  blue   <-- k = 5: blue 2, red 3
distance 3.027  (1.0, 3.0)  red
distance 3.027  (7.0, 3.0)  blue
distance 3.140  (6.5, 4.5)  blue
distance 3.400  (1.0, 1.0)  red
distance 4.020  (8.0, 3.0)  blue
```

Fitting a line

A line `y = w * x + b` is fitted by choosing `w` and `b` that minimise the mean squared error.

output

```
 y
6.8 |                            *
5.6 |                        *
5.1 |                    *
3.9 |            *
2.8 |      *
2.2 | *
    +------------------------------  x
     1    2    3    4    5    6

error for one row   = (w * x + b) - y
mean squared error  = average of error squared over all rows
gradient of w        = average of 2 * error * x
gradient of b        = average of 2 * error
closed form          = covariance(x, y) / variance(x) for w
```

Both routes in Example 2 land on the same line, which is the point: gradient descent is a general recipe, and the closed form is the special case that happens to exist for a straight line.

Scaling columns

If one column is measured in euros and another in years, the euro column decides every distance. Min-max scaling puts every column into `[0, 1]` using `(value - minimum) / (maximum - minimum)`.

Before scaling	After scaling
income dominates distance	income and age contribute comparably
"near" means "same salary"	"near" means "similar in both columns"
one feature carries the model	each feature carries a share

8.3 Examples

Example 1: k-NN on hand-made points, showing the ranked distance table and the verdict for three values of k.

Python

```python
"""k-nearest neighbours on hand-made points in the plane."""

red = [(1.0, 1.0), (2.0, 2.0), (1.0, 3.0), (3.0, 1.0), (2.0, 3.5)]
blue = [(6.0, 1.5), (7.0, 3.0), (5.5, 2.5), (6.5, 4.5), (8.0, 3.0)]
points = [(p, "red") for p in red] + [(p, "blue") for p in blue]

def distance(a, b):
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def classify(query, k):
    ranked = sorted(points, key=lambda item: distance(query, item[0]))
    nearest = ranked[:k]
    votes = {}
    for _point, label in nearest:
        votes[label] = votes.get(label, 0) + 1
    if votes.get("red", 0) == votes.get("blue", 0):
        weights = {}
        for point, label in nearest:
            weights[label] = (weights.get(label, 0.0)
                              + 1.0 / distance(query, point))
        winner = max(weights, key=weights.get)
        return nearest, winner, "tie broken by 1/d weight"
    winner = max(votes, key=votes.get)
    return nearest, winner, ""

query = (4.0, 2.6)
ranked = sorted(points, key=lambda item: distance(query, item[0]))
print("query point: %s" % (query,))
print("%-14s %-6s %s" % ("point", "label", "distance"))
for point, label in ranked:
    print("%-14s %-6s %.3f" % (point, label, distance(query, point)))
print()

for k in (1, 3, 5):
    nearest, winner, note = classify(query, k)
    print("k = %d, neighbours: %s -> %s %s"
          % (k, ", ".join(l for _p, l in nearest), winner, note))

print()
print("with k = 1 the single closest point is blue, so blue wins;")
print("with k = 3 or 5 the red points outvote it")
```

output

```
query point: (4.0, 2.6)
point          label  distance
(5.5, 2.5)     blue   1.503
(3.0, 1.0)     red    1.887
(2.0, 2.0)     red    2.088
(2.0, 3.5)     red    2.193
(6.0, 1.5)     blue   2.283
(1.0, 3.0)     red    3.027
(7.0, 3.0)     blue   3.027
(6.5, 4.5)     blue   3.140
(1.0, 1.0)     red    3.400
(8.0, 3.0)     blue   4.020

k = 1, neighbours: blue -> blue
k = 3, neighbours: blue, red, red -> red
k = 5, neighbours: blue, red, red, red, blue -> red

with k = 1 the single closest point is blue, so blue wins;
with k = 3 or 5 the red points outvote it
```

The nearest neighbour is one blue point that sits slightly on the red side of the query, which is exactly the case a small `k` handles badly. Raising `k` brings in the three red points that surround it, and the verdict flips. This is why `k` is chosen on held-out data rather than by taste.

Example 2: linear regression by gradient descent, checked against the closed-form line.

Python

```
"""Simple linear regression: gradient descent, then the closed form."""

data = [(1, 2.2), (2, 2.8), (3, 3.9), (4, 5.1), (5, 5.6), (6, 6.8)]
n = len(data)

w, b = 0.0, 0.0
lr = 0.02

def mse(w, b):
    return sum((w * x + b - y) ** 2 for x, y in data) / n

print("gradient descent, learning rate %.2f" % lr)
for epoch in range(1, 3001):
    grad_w = sum(2 * (w * x + b - y) * x for x, y in data) / n
    grad_b = sum(2 * (w * x + b - y) for x, y in data) / n
    w -= lr * grad_w
    b -= lr * grad_b
    if epoch % 500 == 0:
        print("  epoch %4d: w = %.4f  b = %.4f  MSE = %.5f"
              % (epoch, w, b, mse(w, b)))

mean_x = sum(x for x, _y in data) / n
mean_y = sum(y for _x, y in data) / n
cov = sum((x - mean_x) * (y - mean_y) for x, y in data)
var = sum((x - mean_x) ** 2 for x, y in data)
exact_w = cov / var
exact_b = mean_y - exact_w * mean_x

print()
print("closed form: w = %.6f, b = %.6f" % (exact_w, exact_b))

ss_res = sum((y - (exact_w * x + exact_b)) ** 2 for x, y in data)
ss_tot = sum((y - mean_y) ** 2 for x, y in data)
print("predictions with the fitted line")
for x, y in data:
    print("  x = %d, observed %.2f, predicted %.2f, residual %+.2f"
          % (x, y, exact_w * x + exact_b, (exact_w * x + exact_b) - y))
print()
print("R^2 = 1 - SSres/SStot = 1 - %.4f / %.4f = %.4f"
      % (ss_res, ss_tot, 1 - ss_res / ss_tot))
```

output

```
gradient descent, learning rate 0.02
  epoch  500: w = 0.9367  b = 1.1176  MSE = 0.02638
  epoch 1000: w = 0.9316  b = 1.1394  MSE = 0.02629
  epoch 1500: w = 0.9314  b = 1.1400  MSE = 0.02629
  epoch 2000: w = 0.9314  b = 1.1400  MSE = 0.02629
  epoch 2500: w = 0.9314  b = 1.1400  MSE = 0.02629
  epoch 3000: w = 0.9314  b = 1.1400  MSE = 0.02629

closed form: w = 0.931429, b = 1.140000
predictions with the fitted line
  x = 1, observed 2.20, predicted 2.07, residual -0.13
  x = 2, observed 2.80, predicted 3.00, residual +0.20
  x = 3, observed 3.90, predicted 3.93, residual +0.03
  x = 4, observed 5.10, predicted 4.87, residual -0.23
  x = 5, observed 5.60, predicted 5.80, residual +0.20
  x = 6, observed 6.80, predicted 6.73, residual -0.07

R^2 = 1 - SSres/SStot = 1 - 0.1577 / 15.3400 = 0.9897
```

Gradient descent starts from `w = 0` and `b = 0` and converges to the closed-form line. The residuals are all below 0.25, their sum is 0, and `R^2` says the line explains about 99 percent of the variation in the observed values. Note the diagnostic habit: print the parameters every few hundred epochs so that a divergence is visible immediately.

Example 3: the same model before and after scaling the columns, on a customer table where one column is measured in euros and the other in years.

Python

```python
"""The same k-NN model before and after feature scaling."""

customers = [
    (32000, 24, "no"),
    (45000, 31, "no"),
    (60000, 29, "yes"),
    (51000, 35, "yes"),
    (38000, 22, "no"),
    (55000, 41, "yes"),
    (48000, 26, "no"),
]
query = (52000, 30)

def distance(a, b):
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def knn(rows, k):
    ranked = sorted(rows, key=lambda row: distance(query, row[:2]))
    nearest = ranked[:k]
    votes = {}
    for row in nearest:
        votes[row[2]] = votes.get(row[2], 0) + 1
    return nearest, max(votes, key=votes.get)

print("unscaled, k = 3")
nearest, winner = knn(customers, 3)
for row in nearest:
    print("  income %6d age %2d label %-3s distance %10.1f"
          % (row[0], row[1], row[2], distance(query, row[:2])))
print("  predicted: %s" % winner)

incomes = [row[0] for row in customers]
ages = [row[1] for row in customers]
lo_i, hi_i = min(incomes), max(incomes)
lo_a, hi_a = min(ages), max(ages)

scaled = [((row[0] - lo_i) / (hi_i - lo_i),
           (row[1] - lo_a) / (hi_a - lo_a), row[2]) for row in customers]
query_scaled = ((query[0] - lo_i) / (hi_i - lo_i),
                (query[1] - lo_a) / (hi_a - lo_a))

def distance_scaled(a, b):
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def knn_scaled(rows, k):
    ranked = sorted(rows, key=lambda row: distance_scaled(query_scaled, row[:2]))
    nearest = ranked[:k]
    votes = {}
    for row in nearest:
        votes[row[2]] = votes.get(row[2], 0) + 1
    return nearest, max(votes, key=votes.get)

print()
print("scaled to [0, 1], k = 3")
nearest, winner = knn_scaled(scaled, 3)
for row in nearest:
```

Python (continued)

```
    print("  income %.3f age %.3f label %-3s distance %.3f"
          % (row[0], row[1], row[2], distance_scaled(query_scaled, row[:2])))
print("  predicted: %s" % winner)
print()
print("without scaling the income column drowns out the age column")
```

output

```
unscaled, k = 3
  income  51000 age 35 label yes distance     1000.0
  income  55000 age 41 label yes distance     3000.0
  income  48000 age 26 label no  distance     4000.0
  predicted: yes

scaled to [0, 1], k = 3
  income 0.571 age 0.211 label no  distance 0.254
  income 0.464 age 0.474 label no  distance 0.255
  income 0.679 age 0.684 label yes distance 0.266
  predicted: no

without scaling the income column drowns out the age column
```

The verdict flipped from yes to no without changing a single label. Unscaled, the three nearest neighbours are chosen almost entirely by income; scaled, the age column contributes and the neighbours change. Neither answer is the true one here, but only the scaled version is the one a person would defend.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Testing on the training rows	k-NN with k = 1 scores 100 percent and learns nothing	always hold rows out
Choosing k without checking	small k chases noise, large k erases minority classes	compare a range of k on held-out rows
Ignoring column scales	the largest column picks the neighbours	scale the columns, or say why not
Fitting a line to curved data	the model is wrong in a way no metric reveals	plot residuals, or fit a curve if they curve
Reading a gradient descent run once	a diverging run looks like a bug	print parameters and loss every few hundred epochs
Treating `R^2` as accuracy	it measures explained variation, not correct labels	use it for regression, and accuracy for classification

- Residuals should look unstructured; any curve in them means the model is missing something.
- Two features with different units are only comparable after scaling, unless you deliberately want one to dominate.

8.5 Practice exercises

1. A k-NN classifier with k = 3 sees neighbours labelled cat, cat and dog. What does it predict, and what would change if the distances were 1.0, 1.1 and 5.0?
2. Why does k = 1 on the training rows always produce a perfect score, and why is that score useless?
3. In Example 2, write the fitted line as an equation and predict the value at `x = 7`.
4. In Example 3, compute the scaled coordinates of the query point by hand, using the income range 28000 and the age range 19, then check one neighbour's printed values against your calculation.
5. Give one reason to prefer linear regression over k-NN, and one to prefer k-NN over linear regression.

Solutions

1. It predicts cat, by a simple majority of two against one. If the votes were tied, weighting by `1 / distance` would settle it, and with distances 1.0, 1.1 and 5.0 the two cats would outweigh the dog even under a weighted rule.
2. Because the closest training row to any row is itself, at distance zero, so the label is copied exactly. The score measures memory, not the ability to generalise, and it collapses the first time unseen rows arrive.
3. The line is `y = 0.931429 * x + 1.14`, so at `x = 7` it predicts `0.931429 * 7 + 1.14 = 7.66`.
4. The query scales to income `(52000 - 32000) / 28000 = 0.7143` and age `(30 - 22) / 19 = 0.4211`. The first printed neighbour shows income 0.571 and age 0.211, which matches `(48000 - 32000) / 28000` and `(26 - 22) / 19`, so the scaling is right.
5. Regression is preferred when the trend is roughly straight, because it gives two readable numbers, predicts instantly and shows its coefficients. k-NN is preferred when the boundary is irregular or unknown, because it makes no assumption about the shape of the boundary at all.

8.6 Mini test

1. What does a k-NN classifier predict for a new row? A) The most common label in the training set B) The majority label among the k closest training rows C) The average of the labels D) The label of the row with the highest income
2. Why should k be odd for a two-class problem? A) Odd numbers are faster B) To avoid an exact tie in the vote C) Because k must be smaller than the data set D) Because distances are squared
3. What does the gradient of the mean squared error point towards? A) The direction that increases the loss the fastest B) The direction that increases the loss the slowest C) The direction that decreases the loss the fastest D) The minimum of the data set
4. True or false: gradient descent and the closed form always give the same numbers for the same data.
5. In Example 3, what removed the dominance of the income column? A) A larger k B) Min-max scaling to [0, 1] C) Removing the income column D) Sorting the rows

6. What does `R^2` measure? A) The share of labels predicted correctly B) The share of variation in the labels explained by the model C) The number of parameters in the model D) The distance to the nearest neighbour

Answer key and explanations

1. B - k-NN is defined by the vote of the closest rows, which is why distance and scaling matter so much.
2. B - with two classes an even k can split the vote, while an odd k always gives a majority.
3. C - moving against the gradient descends the loss, which is the whole of gradient descent.
4. True - for mean squared error on a line, both routes converge to the same minimiser, as the output of Example 2 shows.
5. B - scaling puts every column on a comparable range so distances weigh them together.
6. B - `R^2` compares the residual sum of squares with the total sum of squares of the labels.

Chapter 9 - Neural network basics

9.1 Summary

- A neuron sums its inputs after weighting each one, adds a bias, and passes the total through an activation function.
- With a step activation, a single neuron is a linear classifier that draws one straight boundary.
- The perceptron learning rule adjusts weights only when a row is misclassified, and converges on separable data.
- A single perceptron cannot learn XOR, because no straight line separates the four corners of the truth table.
- A hidden layer creates new features, so a two-layer network can learn the functions a single neuron cannot.
- The sigmoid squashes any value into a range between 0 and 1 and has a derivative that is easy to compute.
- Backpropagation applies the chain rule from the output error backwards through the weights of each layer.
- Every weight receives a gradient that is the product of the error signal and the activation that fed it.
- A learning rate that is too large overshoots and diverges, while one that is too small barely moves.
- A finite-difference check of one gradient proves the chain rule was carried out correctly, as Example 2 shows.

9.2 Concepts

One neuron

output

```
                    weights
         x1 ---------------- w1 -----\
         x2 ---------------- w2 -------+--> z = w1x1 + w2x2 + b --> a = f(z)
         bias -------------- b -------/

   f = step    : a = 1 if z >= 0 else 0        (a perceptron)
   f = sigmoid : a = 1 / (1 + exp(-z))         (smooth, differentiable)
```

The truth tables that matter

Inputs	AND	OR	XOR
0, 0	0	0	0
0, 1	0	1	1
1, 0	0	1	1
1, 1	1	1	0

The XOR column is the interesting one: the rows that output 1 sit on opposite corners, so no single straight boundary separates them.

output

```
    x2
   1 |  .(0,1)=1        .(1,1)=0       no straight line can put
     |                                   the two 1 corners together
   0 |  .(0,0)=0        .(1,0)=1
     +-----------------------------  x1
       0                     1
```

Why a hidden layer helps

A hidden unit computes a function of the inputs, and the output unit can then combine those functions. Two hidden units can, for instance, separate the XOR corners by reacting to different combinations of inputs.

output

```
    inputs                 hidden                 output
   +-----+  w1  +----+    +----+   w3    +-----+
   | x1  |----->| z1 |--->| h1 |-------->|     |
   +-----+      +----+    +----+         |  z  |----> y
   +-----+  w2  +----+    +----+   w4    |     |
   | x2  |----->| z2 |--->| h2 |-------->|     |
   +-----+      +----+    +----+         +-----+
   each hidden unit draws its own boundary; the output combines them
```

Backpropagation in one line

The error at the output is spread backwards along each path, and the gradient of a weight is the product of the signal and the activation that arrived at it.

output

```
  d loss / d w = (d loss / d y) * (d y / d z out) * (d z out / d w)
                   ^ error at output   ^ derivative of   ^ the activation
                                       the activation     that fed the weight
```

9.3 Examples

Example 1: one perceptron, trained three times on AND, on OR and on XOR.

Python

```python
"""A single perceptron trained with the perceptron learning rule."""

def step(z):
    return 1 if z >= 0 else 0

AND = [((0, 0), 0), ((0, 1), 0), ((1, 0), 0), ((1, 1), 1)]
OR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 1)]
XOR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 0)]

def train(pairs, lr=0.1, epochs=20):
    w = [0.5, -0.5]        # fixed starting weights, no randomness here
    b = -0.25
    log = []
    for epoch in range(epochs):
        errors = 0
        for x, target in pairs:
            guess = step(w[0] * x[0] + w[1] * x[1] + b)
            error = target - guess
            if error:
                errors += 1
                w[0] += lr * error * x[0]
                w[1] += lr * error * x[1]
                b += lr * error
        log.append((epoch + 1, errors, w[0], w[1], b))
        if errors == 0:
            break
    return w, b, log

for name, data in (("AND", AND), ("OR", OR), ("XOR", XOR)):
    w, b, log = train(data)
    print("%s: stopped after %d epoch(s)" % (name, len(log)))
    for epoch, errors, w1, w2, bias in log:
        print("   epoch %2d: %d mistake(s), w1 = %+.2f, w2 = %+.2f, b = %+.2f"
              % (epoch, errors, w1, w2, bias))
    for x, target in data:
        guess = step(w[0] * x[0] + w[1] * x[1] + b)
        print("   %s -> %d (target %d) %s"
              % (x, guess, target, "ok" if guess == target else "WRONG"))
    print()
```

output

```
AND: stopped after 10 epoch(s)
   epoch  1: 2 mistake(s), w1 = +0.50, w2 = -0.40, b = -0.25
   epoch  2: 2 mistake(s), w1 = +0.50, w2 = -0.30, b = -0.25
   epoch  3: 2 mistake(s), w1 = +0.50, w2 = -0.20, b = -0.25
   epoch  4: 2 mistake(s), w1 = +0.50, w2 = -0.10, b = -0.25
   epoch  5: 2 mistake(s), w1 = +0.50, w2 = -0.00, b = -0.25
   epoch  6: 1 mistake(s), w1 = +0.40, w2 = -0.00, b = -0.35
   epoch  7: 2 mistake(s), w1 = +0.40, w2 = +0.10, b = -0.35
   epoch  8: 2 mistake(s), w1 = +0.40, w2 = +0.20, b = -0.35
   epoch  9: 1 mistake(s), w1 = +0.30, w2 = +0.20, b = -0.45
   epoch 10: 0 mistake(s), w1 = +0.30, w2 = +0.20, b = -0.45
   (0, 0) -> 0 (target 0) ok
   (0, 1) -> 0 (target 0) ok
   (1, 0) -> 0 (target 0) ok
   (1, 1) -> 1 (target 1) ok

OR: stopped after 7 epoch(s)
   epoch  1: 2 mistake(s), w1 = +0.60, w2 = -0.30, b = -0.05
   epoch  2: 1 mistake(s), w1 = +0.60, w2 = -0.20, b = +0.05
   epoch  3: 2 mistake(s), w1 = +0.60, w2 = -0.10, b = +0.05
   epoch  4: 2 mistake(s), w1 = +0.60, w2 = -0.00, b = +0.05
   epoch  5: 2 mistake(s), w1 = +0.60, w2 = +0.10, b = +0.05
   epoch  6: 1 mistake(s), w1 = +0.60, w2 = +0.10, b = -0.05
   epoch  7: 0 mistake(s), w1 = +0.60, w2 = +0.10, b = -0.05
   (0, 0) -> 0 (target 0) ok
   (0, 1) -> 1 (target 1) ok
   (1, 0) -> 1 (target 1) ok
   (1, 1) -> 1 (target 1) ok

XOR: stopped after 20 epoch(s)
   epoch  1: 1 mistake(s), w1 = +0.50, w2 = -0.40, b = -0.15
   epoch  2: 2 mistake(s), w1 = +0.40, w2 = -0.40, b = -0.15
   epoch  3: 2 mistake(s), w1 = +0.30, w2 = -0.40, b = -0.15
   epoch  4: 1 mistake(s), w1 = +0.30, w2 = -0.30, b = -0.05
   epoch  5: 2 mistake(s), w1 = +0.20, w2 = -0.30, b = -0.05
   epoch  6: 2 mistake(s), w1 = +0.10, w2 = -0.30, b = -0.05
   epoch  7: 1 mistake(s), w1 = +0.10, w2 = -0.20, b = +0.05
   epoch  8: 3 mistake(s), w1 = +0.00, w2 = -0.20, b = -0.05
   epoch  9: 1 mistake(s), w1 = +0.00, w2 = -0.10, b = +0.05
   epoch 10: 3 mistake(s), w1 = -0.10, w2 = -0.10, b = -0.05
   epoch 11: 3 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 12: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 13: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 14: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 15: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 16: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 17: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 18: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 19: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   epoch 20: 4 mistake(s), w1 = -0.10, w2 = -0.10, b = +0.05
   (0, 0) -> 1 (target 0) WRONG
   (0, 1) -> 0 (target 1) WRONG
   (1, 0) -> 0 (target 1) WRONG
   (1, 1) -> 0 (target 0) ok
```

AND converges after ten epochs and OR after seven, both reaching zero mistakes. XOR never does: the weights stop moving at epoch 12 and stay wrong on three of the four rows, which is what a non-separable truth table does to a single straight boundary. The failure is not a bug in the code but a fact about the model.

Example 2: one forward and backward pass, printed number by number, with a finite-difference check of the gradient.

Python

```python
"""One forward and backward pass of a 2-2-1 network, printed number by number."""

import math

def sigmoid(z):
    return 1.0 / (1.0 + math.exp(-z))

x = [1.0, 0.5]
W1 = [[0.5, -0.4], [0.3, 0.8]]
b1 = [0.1, -0.2]
W2 = [[0.7], [-0.3]]
b2 = [0.05]
target = 1.0

def forward(params):
    W1, b1, W2, b2 = params
    h_in = []
    for i in range(2):
        h_in.append(W1[i][0] * x[0] + W1[i][1] * x[1] + b1[i])
    h = [sigmoid(z) for z in h_in]
    y_in = W2[0][0] * h[0] + W2[1][0] * h[1] + b2[0]
    y = sigmoid(y_in)
    return h_in, h, y_in, y

base = (W1, b1, W2, b2)
h_in, h, y_in, y = forward(base)
loss = 0.5 * (target - y) ** 2

print("forward pass")
print("  z1 = [%.4f, %.4f]" % (h_in[0], h_in[1]))
print("  h  = [%.4f, %.4f]" % (h[0], h[1]))
print("  z2 = %.4f, y = %.4f" % (y_in, y))
print("  loss = 0.5 * (1 - %.4f)^2 = %.6f" % (y, loss))

d_out = (y - target) * y * (1 - y)
grad_W2 = [d_out * h[0], d_out * h[1]]
grad_b2 = d_out
d_hidden = [d_out * W2[i][0] * h[i] * (1 - h[i]) for i in range(2)]
grad_W1 = [[d_hidden[i] * x[j] for j in range(2)] for i in range(2)]
grad_b1 = d_hidden[:]

print()
print("backward pass")
print("  d_y/d_loss  = -(target - y) = %.4f" % (target - y))
print("  delta2 = (y - t) * y(1-y) = %.2f * %.4f = %.6f"
      % (y - target, y * (1 - y), d_out))
print("  grad W2 = [%.6f, %.6f], grad b2 = %.6f"
      % (grad_W2[0], grad_W2[1], grad_b2))
print("  delta1 = [%.6f, %.6f]" % (d_hidden[0], d_hidden[1]))
print("  grad W1 = [[%.6f, %.6f], [%.6f, %.6f]]"
      % (grad_W1[0][0], grad_W1[0][1], grad_W1[1][0], grad_W1[1][1]))
print("  grad b1 = [%.6f, %.6f]" % (grad_b1[0], grad_b1[1]))

"""A numeric gradient check for one weight."""
eps = 1e-6
tweaked = ([row[:] for row in W1], b1[:], W2[:], b2[:])
```

Python (continued)

```
tweaked[0][0][0] += eps
loss_tweaked = 0.5 * (target - forward(tweaked)[3]) ** 2
numeric = (loss_tweaked - loss) / eps
print()
print("numeric check of d loss / d W1[0][0]")
print("  chain rule : %.6f" % grad_W1[0][0])
print("  finite diff : %.6f" % numeric)
```

output

```
forward pass
  z1 = [0.4000, 0.5000]
  h  = [0.5987, 0.6225]
  z2 = 0.2823, y = 0.5701
  loss = 0.5 * (1 - 0.5701)^2 = 0.092398

backward pass
  d_y/d_loss  = -(target - y) = 0.4299
  delta2 = (y - t) * y(1-y) = -0.43 * 0.2451 = -0.105356
  grad W2 = [-0.063075, -0.065580], grad b2 = -0.105356
  delta1 = [-0.017719, 0.007428]
  grad W1 = [[-0.017719, -0.008860], [0.007428, 0.003714]]
  grad b1 = [-0.017719, 0.007428]

numeric check of d loss / d W1[0][0]
  chain rule : -0.017719
  finite diff : -0.017719
```

Every line can be recomputed by hand. The hidden values are `sigmoid(0.4) = 0.5987` and `sigmoid(0.5) = 0.6225`; the output value is `sigmoid(0.7 * 0.5987 - 0.3 * 0.6225 + 0.05) = sigmoid(0.2823) = 0.5701`. The two hidden deltas differ in sign because the two output weights have opposite signs, which is why `delta1` is `[-0.017719, 0.007428]` rather than two equal numbers.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Expecting one neuron to solve XOR	the truth table is not separable by a line	add a hidden layer
Updating weights with the raw error	the step is too large and the rule oscillates	scale by a learning rate
Forgetting the derivative of the activation	the chain rule stops at the wrong place	write the gradient as a product of three factors
Skipping the finite-difference check	a sign error hides for hours	compare one gradient against a numeric estimate
Using a step activation in a hidden layer	the derivative is zero almost everywhere	use a smooth activation such as the sigmoid
Reading a deep network as understandable	the hidden units are features nobody designed	inspect them, as the next chapter does

- If a network will not learn, first check the loss on the first ten epochs: it should fall, however slowly.
- When the loss falls and the predictions are wrong, the problem is the model or the data, not the arithmetic.

9.5 Practice exercises

1. Compute by hand the output of a perceptron with `w1 = 0.3`, `w2 = 0.2`, `b = -0.45` for all four XOR inputs, and compare with the AND result.
2. Why does the XOR run in Example 1 stop changing at epoch 12?
3. In Example 2, compute `sigmoid(0.4)` and `sigmoid(-0.3)` and check them against the printed values.
4. What does the finite-difference check prove, and what does it not prove?
5. A network fails to learn at all, and the loss stays at 0.125 after 6000 epochs. Name three possible causes.

Solutions

1. `z = 0.3 * x1 + 0.2 * x2 - 0.45`, so the outputs are 0, 0, 0 and `0.3 + 0.2 - 0.45 = 0.05`, which gives 1. That is exactly the AND column, and it is wrong three times on XOR.
2. Because after epoch 12 the weights sit where every row is misclassified in a way that cancels: the rule pushes the boundary in one direction for one pair of rows and back for the other, so the weights stop changing and the mistakes stay at four.
3. `sigmoid(0.4) = 1 / (1 + exp(-0.4)) = 0.5987`, and `sigmoid(-0.3) = 1 / (1 + exp(0.3)) = 0.4256`. The first matches the printed hidden value; the second does not appear in the output, since the network never applies the sigmoid to `-0.3`.
4. It proves that the analytic gradient of the loss for that one weight is correct, so the chain rule was applied correctly at least on that path. It does not prove the whole network is correct, that the learning rate is sensible, or that the model can fit the data.
5. The learning rate is far too small; the starting weights are unlucky and the network sits in a flat region; or the labels are wrong, for example the targets are inverted, so no setting of weights can lower the loss.

9.6 Mini test

1. What does a neuron compute before its activation function? A) The average of the inputs B) A weighted sum of the inputs plus a bias C) The largest input D) The product of all inputs
2. Which activation makes a perceptron a linear classifier? A) The sigmoid B) The step function C) The absolute value D) The square
3. Why can one perceptron not learn XOR? A) XOR has four inputs B) No single straight boundary separates its two classes C) The perceptron has no bias D) XOR needs negative weights
4. True or false: a hidden layer changes the functions available to the network.

5. What is the derivative of the sigmoid, expressed using its own output? A) `a * (1 - a)` B) `1 - a` C) `a` D) `1 / a`
6. In the backward pass, why do the two hidden deltas have opposite signs? A) Because the two inputs are different B) Because the two output weights have opposite signs C) Because one hidden unit is broken D) Because the bias is negative

Answer key and explanations

1. B - the pre-activation is `w1 * x1 + w2 * x2 + b`, which is then passed through `f`.
2. B - the step function turns a straight boundary into a yes or no decision, which is a linear classifier.
3. B - the two classes of XOR occupy opposite corners, so no line can separate them, and the perceptron cannot converge.
4. True - hidden units form new features from the inputs, which is what gives the network its extra expressiveness.
5. A - the sigmoid's derivative is `a(1 - a)`, one of the reasons it was popular before other activations.
6. B - each hidden delta contains its own output weight, so opposite weights push the two hidden units in opposite directions.

Chapter 10 - Building and training a network

10.1 Summary

- A two-layer network with sigmoid activations learns AND, OR and XOR, which a single neuron cannot.
- Training is a loop: forward pass, compare with the target, backpropagate, adjust every weight.
- Full-batch gradient descent averages the gradients of all four rows before each step, so no randomness is needed.
- The learning rate sets the step size; too small stalls, too large diverges.
- A fixed seed makes the starting weights reproducible, which is why the printed losses can be checked line by line.
- The loss curve is the main diagnostic: a falling curve means learning, a flat one means stalling, a rising one means divergence.
- Some starting points never leave the flat part on XOR, which is why seeds are compared rather than trusted.
- After training, the hidden activations reveal the representation the network invented on its own.
- A network learns the function it was trained on and nothing else: the XOR network fails on AND and on OR.

- Ordinary Python floats are enough to build and train a small network, with no library at all.

10.2 Concepts

The training loop

output

```
+-----------+      +-----------+      +------------+      +----------+
| starting  | ---> | forward   | ---> | loss       | ---> | backward |
| weights   |      | pass      |      | 0.5(y-t)^2 |      | pass     |
+-----------+      +-----------+      +------------+      +----------+
     ^                                                        |
     |            weights = weights - lr * gradient           |
     +--------------------------------------------------------+
                           repeat for many epochs
```

The ingredients, and what each one controls

Ingredient	In the example	What it controls
Weights and biases	drawn from a seeded generator	where the search starts
Learning rate	0.5	the size of each step
Epochs	6000	how many passes over the data
Loss	mean squared error over four rows	what "better" means
Gradient	averaged over all four rows	the direction of the step

Learning rates compared on the same task

Learning rate	Behaviour
0.1 with 3000 epochs	the loss stalls near 0.12 and only 2 to 3 of 4 rows are right: undertrained
1.0	loss falls below 0.003 and all four rows are right
5.0	the loss falls fastest, though a larger rate is usually the first place to look when training is unstable

The table is printed by Example 2 for three seeds each. The slow rate is not a bug: the sum of squared errors is small in the middle of the sigmoid's range, so the gradient is small too, and a small step multiplied by a small gradient barely moves.

What the hidden layer learns

For XOR, the four inputs need four distinguishable hidden patterns, because a single boundary cannot separate them.

x1, x2	hidden 1	hidden 2	output
0, 0	high	high	low

x1, x2	hidden 1	hidden 2	output
0, 1	medium	low	high
1, 0	medium	low	high
1, 1	low	very low	low

The two middle rows have nearly the same hidden value in the first unit, which the second unit and the output weights separate. The network did not invent these features to be readable; it found numbers that lowered the loss.

10.3 Examples

Example 1: a two-layer network trained on XOR, then tested on XOR, AND and OR.

Python

```python
"""A 2-layer network (2 inputs, 2 hidden, 1 output) trained on XOR."""

import math
import random

def sigmoid(z):
    if z >= 0:
        return 1.0 / (1.0 + math.exp(-z))
    e = math.exp(z)
    return e / (1.0 + e)

def make_net(seed):
    rng = random.Random(seed)
    W1 = [[rng.uniform(-1, 1) for _ in range(2)] for _ in range(2)]
    b1 = [rng.uniform(-1, 1) for _ in range(2)]
    W2 = [[rng.uniform(-1, 1)] for _ in range(2)]
    b2 = [rng.uniform(-1, 1)]
    return W1, b1, W2, b2

def forward(net, x):
    W1, b1, W2, b2 = net
    h = [sigmoid(W1[i][0] * x[0] + W1[i][1] * x[1] + b1[i]) for i in range(2)]
    y = sigmoid(W2[0][0] * h[0] + W2[1][0] * h[1] + b2[0])
    return h, y

def train(net, data, lr, epochs):
    W1, b1, W2, b2 = net
    for epoch in range(epochs):
        gW1 = [[0.0] * 2 for _ in range(2)]
        gb1 = [0.0, 0.0]
        gW2 = [0.0, 0.0]
        gb2 = 0.0
        total_loss = 0.0
        for x, target in data:
            h, y = forward(net, x)
            error = y - target
            total_loss += 0.5 * error * error
            delta2 = error * y * (1 - y)
            gW2[0] += delta2 * h[0]
            gW2[1] += delta2 * h[1]
            gb2 += delta2
            for i in range(2):
                d1 = delta2 * W2[i][0] * h[i] * (1 - h[i])
                gb1[i] += d1
                for j in range(2):
                    gW1[i][j] += d1 * x[j]
        n = len(data)
        for i in range(2):
            W2[i][0] -= lr * gW2[i] / n
            b1[i] -= lr * gb1[i] / n
            for j in range(2):
                W1[i][j] -= lr * gW1[i][j] / n
        b2[0] -= lr * gb2 / n
        if epoch == 0 or (epoch + 1) % 1000 == 0:
            print("  epoch %5d  mean loss %.6f" % (epoch + 1, total_loss / n))
    return net
```

Python (continued)

```

XOR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 0)]
AND = [((0, 0), 0), ((0, 1), 0), ((1, 0), 0), ((1, 1), 1)]
OR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 1)]

print("training on XOR with seed 42, lr 0.5, 6000 epochs")
net = train(make_net(42), XOR, 0.5, 6000)
print()
for name, data in (("XOR", XOR), ("AND", AND), ("OR", OR)):
    correct = 0
    print("%s" % name)
    for x, target in data:
        _h, y = forward(net, x)
        ok = (y >= 0.5) == bool(target)
        correct += ok
        print("  x = %s target %d output %.4f -> %d %s"
              % (x, target, y, 1 if y >= 0.5 else 0,
                 "ok" if ok else "WRONG"))
    print("  %d/4 correct" % correct)
```

output

```
training on XOR with seed 42, lr 0.5, 6000 epochs
  epoch     1  mean loss 0.125626
  epoch  1000  mean loss 0.118101
  epoch  2000  mean loss 0.080066
  epoch  3000  mean loss 0.020292
  epoch  4000  mean loss 0.006404
  epoch  5000  mean loss 0.003408
  epoch  6000  mean loss 0.002250

XOR
  x = (0, 0) target 0 output 0.0564 -> 0 ok
  x = (0, 1) target 1 output 0.9364 -> 1 ok
  x = (1, 0) target 1 output 0.9361 -> 1 ok
  x = (1, 1) target 0 output 0.0818 -> 0 ok
  4/4 correct
AND
  x = (0, 0) target 0 output 0.0564 -> 0 ok
  x = (0, 1) target 0 output 0.9364 -> 1 WRONG
  x = (1, 0) target 0 output 0.9361 -> 1 WRONG
  x = (1, 1) target 1 output 0.0818 -> 0 WRONG
  1/4 correct
OR
  x = (0, 0) target 0 output 0.0564 -> 0 ok
  x = (0, 1) target 1 output 0.9364 -> 1 ok
  x = (1, 0) target 1 output 0.9361 -> 1 ok
  x = (1, 1) target 1 output 0.0818 -> 0 WRONG
  3/4 correct
```

The loss falls from 0.1256 to 0.00225 and all four XOR rows are right. The same network then fails on AND and OR, which is not a contradiction: the weights encode the shape of XOR, and the other functions have a different shape. This is the clearest possible statement of what "training" means.

Example 2: the same task with three seeds and three learning rates, plus a reproducibility check.

Python

```python
"""The same XOR task, different random starts and learning rates."""

import math
import random

def sigmoid(z):
    if z >= 0:
        return 1.0 / (1.0 + math.exp(-z))
    e = math.exp(z)
    return e / (1.0 + e)

XOR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 0)]

def make_net(seed):
    rng = random.Random(seed)
    W1 = [[rng.uniform(-1, 1) for _ in range(2)] for _ in range(2)]
    b1 = [rng.uniform(-1, 1) for _ in range(2)]
    W2 = [[rng.uniform(-1, 1)] for _ in range(2)]
    b2 = [rng.uniform(-1, 1)]
    return W1, b1, W2, b2

def run(seed, lr, epochs=3000):
    W1, b1, W2, b2 = make_net(seed)
    for _epoch in range(epochs):
        gW1 = [[0.0] * 2 for _ in range(2)]
        gb1 = [0.0, 0.0]
        gW2 = [0.0, 0.0]
        gb2 = 0.0
        total = 0.0
        for x, target in XOR:
            h = [sigmoid(W1[i][0] * x[0] + W1[i][1] * x[1] + b1[i])
                 for i in range(2)]
            y = sigmoid(W2[0][0] * h[0] + W2[1][0] * h[1] + b2[0])
            error = y - target
            total += 0.5 * error * error
            delta2 = error * y * (1 - y)
            gW2[0] += delta2 * h[0]
            gW2[1] += delta2 * h[1]
            gb2 += delta2
            for i in range(2):
                d1 = delta2 * W2[i][0] * h[i] * (1 - h[i])
                gb1[i] += d1
                for j in range(2):
                    gW1[i][j] += d1 * x[j]
        for i in range(2):
            W2[i][0] -= lr * gW2[i] / 4
            b1[i] -= lr * gb1[i] / 4
            for j in range(2):
                W1[i][j] -= lr * gW1[i][j] / 4
        b2[0] -= lr * gb2 / 4
    correct = 0
    for x, target in XOR:
        h = [sigmoid(W1[i][0] * x[0] + W1[i][1] * x[1] + b1[i])
             for i in range(2)]
        y = sigmoid(W2[0][0] * h[0] + W2[1][0] * h[1] + b2[0])
        correct += (y >= 0.5) == bool(target)
```

Python (continued)

```
    return total / 4, correct

print("3000 epochs, full-batch gradient descent")
print("%-6s %-6s %-14s %s" % ("seed", "lr", "final loss", "correct"))
for seed in (0, 1, 42):
    for lr in (0.1, 1.0, 5.0):
        loss, correct = run(seed, lr)
        print("%-6d %-6.1f %-14.6f %d/4" % (seed, lr, loss, correct))

print()
a = run(42, 0.5)
b = run(42, 0.5)
print("seed 42, lr 0.5, run twice: %s and %s" % (a, b))
print("same seed, same result:", a == b)
```

output

```
3000 epochs, full-batch gradient descent
seed   lr     final loss     correct
0      0.1    0.123894       3/4
0      1.0    0.003131       4/4
0      5.0    0.000213       4/4
1      0.1    0.124318       3/4
1      1.0    0.001803       4/4
1      5.0    0.000196       4/4
42     0.1    0.123601       2/4
42     1.0    0.002251       4/4
42     5.0    0.000207       4/4

seed 42, lr 0.5, run twice: (0.020292009658505895, 4) and (0.020292009658505895, 4)
same seed, same result: True
```

Three seeds and three learning rates give nine runs. At a learning rate of 0.1 all three seeds stall near a loss of 0.124 with two or three correct rows, so the failure is a property of the setup rather than of one unlucky start. The two faster rates solve all four rows in every seed. Running the same seed twice prints the same loss to the last digit, which is what a seeded generator buys you.

Example 3: the hidden layer after training, printed input by input.

Python

```python
"""What did the hidden layer learn? Print the internal representation."""

import math
import random

def sigmoid(z):
    if z >= 0:
        return 1.0 / (1.0 + math.exp(-z))
    e = math.exp(z)
    return e / (1.0 + e)

XOR = [((0, 0), 0), ((0, 1), 1), ((1, 0), 1), ((1, 1), 0)]

rng = random.Random(42)
W1 = [[rng.uniform(-1, 1) for _ in range(2)] for _ in range(2)]
b1 = [rng.uniform(-1, 1) for _ in range(2)]
W2 = [[rng.uniform(-1, 1)] for _ in range(2)]
b2 = [rng.uniform(-1, 1)]

for _epoch in range(6000):
    gW1 = [[0.0] * 2 for _ in range(2)]
    gb1 = [0.0, 0.0]
    gW2 = [0.0, 0.0]
    gb2 = 0.0
    for x, target in XOR:
        h = [sigmoid(W1[i][0] * x[0] + W1[i][1] * x[1] + b1[i])
             for i in range(2)]
        y = sigmoid(W2[0][0] * h[0] + W2[1][0] * h[1] + b2[0])
        error = y - target
        delta2 = error * y * (1 - y)
        gW2[0] += delta2 * h[0]
        gW2[1] += delta2 * h[1]
        gb2 += delta2
        for i in range(2):
            d1 = delta2 * W2[i][0] * h[i] * (1 - h[i])
            gb1[i] += d1
            for j in range(2):
                gW1[i][j] += d1 * x[j]
    for i in range(2):
        W2[i][0] -= 0.5 * gW2[i] / 4
        b1[i] -= 0.5 * gb1[i] / 4
        for j in range(2):
            W1[i][j] -= 0.5 * gW1[i][j] / 4
    b2[0] -= 0.5 * gb2 / 4

print("%-8s %-8s %-10s %-10s %-8s" % ("x1", "x2", "hidden 1", "hidden 2", "output"))
activations = []
for x, target in XOR:
    h = [sigmoid(W1[i][0] * x[0] + W1[i][1] * x[1] + b1[i])
         for i in range(2)]
    y = sigmoid(W2[0][0] * h[0] + W2[1][0] * h[1] + b2[0])
    activations.append((x, h, y))
    print("%-8d %-8d %-10.4f %-10.4f %-8.4f" % (x[0], x[1], h[0], h[1], y))

print()
print("the four inputs give four different hidden patterns,")
```

Python (continued)

```python
print("which is what a single straight line cannot separate")
```

output

```
x1       x2       hidden 1   hidden 2   output
0        0        0.9963     0.8833     0.0564
0        1        0.8564     0.0293     0.9364
1        0        0.8581     0.0314     0.9361
1        1        0.1186     0.0001     0.0818

the four inputs give four different hidden patterns,
which is what a single straight line cannot separate
```

The second hidden unit is the interesting one: it fires strongly only on the input `(0, 0)` and is almost off for the other three, so it behaves like a "both inputs are zero" detector. The first unit is high for three of the four inputs and drops on `(1, 1)`, and the two units together give the four inputs distinct codes. The output weights then turn those codes into the XOR answer.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Running once and believing the result	one seed can land in a flat region	run several seeds and report the spread
Choosing a learning rate by habit	0.1 stalls here while 1.0 works	compare rates on one fixed seed first
Trusting a loss that stops falling	the network has stalled, not converged	print the loss every few hundred epochs
Judging the model on other functions	a network learns one shape	test it on the function it was trained for
Skipping the seed record	the numbers cannot be reproduced	always print the seed next to the result
Hiding predictions behind one accuracy	four rows have four kinds of error	print every row with its output value

- The loss curve is the whole story of a small training run: read it before anything else.
- When the outputs are confidently wrong, check that the targets are what you think they are.

10.5 Practice exercises

1. In Example 1, what would change if the learning rate were 5.0 instead of 0.5, and what would you look at to decide whether that was an improvement?
2. Why does the loss at epoch 1 equal 0.125626 rather than something smaller?
3. The AND score is 1 of 4. Explain in one sentence why this does not mean the training code is broken.
4. Estimate how many weight updates the training loop of Example 1 performs in total, and show the calculation.
5. Suggest two changes to the training loop that would usually make learning faster or more stable on larger problems.

Solutions

1. The step size grows tenfold, so the same number of epochs moves further; the loss would probably be lower and the XOR rows still right, but the check is the loss curve and the fact that it does not oscillate or grow.
2. With random starting weights near zero, every output is near 0.5, and the target is 0 or 1; the mean squared error is therefore close to `0.5 * (0.5)^2 = 0.125`, which is exactly the printed value.
3. Because the network was trained to fit XOR, and AND has a different shape; a model is a description of the task it was fitted on.
4. There are 10 weights and biases adjusted per epoch, over 6000 epochs, so 60000 updates, each one using the average gradient of four rows.
5. Normalise or scale the inputs, and split the data into smaller batches rather than averaging the whole set each time; both change the gradient's quality rather than its size, and are the usual first levers.

10.6 Mini test

1. What does full-batch gradient descent average before taking a step? A) The weights B) The gradients of all training rows C) The hidden activations D) The learning rates
2. Why is a random seed recorded in an experiment? A) To make the computer faster B) So the starting weights, and therefore the result, can be reproduced C) To remove the need for training D) To make the loss smaller
3. Which loss curve signals a stalled network? A) A curve that keeps falling B) A curve that falls a little and then stays flat C) A curve that rises steadily D) A curve that jumps every epoch
4. True or false: a network trained on XOR can be expected to compute AND without further training.
5. What does the second hidden unit in Example 3 mostly detect? A) A single input being exactly 1 B) Both inputs being 0 C) The target value D) The learning rate
6. Why did every seed stall at a learning rate of 0.1 in Example 2? A) The seed was bad B) The step was too small for the size of the loss in that region C) The loss function was wrong D) The data was shuffled

Answer key and explanations

1. B - gradients are accumulated over all rows and divided by the number of rows before the weights move.
2. B - a seed fixes the starting weights, and the reported numbers are only meaningful if the run can be repeated.
3. B - a flat loss means the updates no longer reduce the error, which is stalling rather than convergence.
4. False - the weights encode one function, and other functions need their own training.
5. B - the unit's activation is high only for the input `(0, 0)` and near zero elsewhere.

6. B - in the middle of the sigmoid's range the error signal is small, so a small rate times a small gradient moves the weights very little.

Chapter 11 - Practical AI with Python

11.1 Summary

- A practical project is a pipeline: prepare the data, split it, fit, predict, measure, then argue about the numbers.
- The confusion matrix is the starting point, and every other metric is read off it.
- Accuracy hides class imbalance, so precision and recall are printed next to it.
- Cross-validation tests every row exactly once, which matters most when data is scarce.
- A fixed seed makes the split and therefore the score reproducible, which is a requirement rather than a nicety.
- Three habits keep results honest: compute the baseline first, report metrics per class, and inspect the errors.
- The metrics in this volume are written in plain Python so that the arithmetic is visible.
- Standard libraries provide the same methods at speed; knowing what they do internally is what makes them usable.
- A short written record of data, seed, split, parameters and metric lets any reader repeat the run.
- Printing a small table of numbers is often enough to see what went wrong, without any plotting.

11.2 Concepts

The pipeline

output

```
raw rows
   |
   v
split by a rule          train 8 rows, test 4 rows
   |
   +--> fit on train     here: store the rows, no parameters at all
   |
   v
predict on test          k nearest neighbours among the 8 train rows
   |
   v
compare with truth       confusion matrix, then metrics
   |
   v
write it down            data, seed, k, split, metrics, mistakes
```

Reading the confusion matrix

The rows are the truth, the columns are the prediction, and each cell counts rows.

Predicted healthy	Predicted at risk	
2	1	truth healthy
0	1	truth at risk

From these four numbers come the standard metrics.

Metric	Formula	Here
Accuracy	correct / total	3/4 = 0.75
Precision	true positive / (true positive + false positive)	1/2 = 0.5
Recall	true positive / (true positive + false negative)	1/1 = 1.0
F1	the harmonic mean of precision and recall	0.667

Precision answers "when the model shouts, how often is it right", and recall answers "of the cases that matter, how many does it find". A model can have recall 1.0 by calling everything at risk, and precision 1.0 by calling almost nothing at risk.

Cross-validation by hand

Step	Action
1	number the rows and split them into folds, here every fourth row
2	hold one fold out, train on the rest, measure on the held-out fold
3	repeat for every fold and collect the scores
4	report the mean, and the spread if the folds disagree

What to record for every run

Item	Why
the data and its size	results are meaningless without the rows
the seed	so the split and any randomness repeat
the split rule	so the test rows are known exactly
the parameters, such as k	so the model can be rebuilt
the metrics and the mistakes	so the next step is obvious

11.3 Examples

Example 1: the whole pipeline in one script, with a confusion matrix and the metrics read from it.

Python

```python
"""A tiny end-to-end pipeline: split, k-NN, then a confusion matrix."""

plants = [
    (2, 45, 10, "healthy"),
    (1, 40,  5, "healthy"),
    (3, 50, 12, "healthy"),
    (7, 80, 25, "at_risk"),
    (8, 85, 20, "at_risk"),
    (6, 75, 30, "at_risk"),
    (4, 58, 18, "healthy"),
    (5, 68, 22, "at_risk"),
    (9, 90, 18, "at_risk"),
    (5, 66, 20, "healthy"),
    (3, 48,  9, "healthy"),
    (7, 82, 28, "at_risk"),
]

train = [row for i, row in enumerate(plants) if i % 3 != 0]
test = [row for i, row in enumerate(plants) if i % 3 == 0]

def squared_distance(a, b):
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2 + (a[2] - b[2]) ** 2

def predict(row, k=3):
    ranked = sorted(train, key=lambda r: squared_distance(r, row))
    votes = {}
    for neighbour in ranked[:k]:
        votes[neighbour[3]] = votes.get(neighbour[3], 0) + 1
    return max(votes, key=votes.get)

truth = [row[3] for row in test]
guess = [predict(row) for row in test]

labels = ["healthy", "at_risk"]
print("confusion matrix, rows = truth, columns = prediction")
print("%-12s %10s %10s" % ("", "pred " + labels[0], "pred " + labels[1]))
counts = {}
for t, g in zip(truth, guess):
    counts[(t, g)] = counts.get((t, g), 0) + 1
for t in labels:
    print("%-12s %10d %10d" % ("true " + t, counts.get((t, labels[0]), 0),
                               counts.get((t, labels[1]), 0)))

total = len(test)
correct = sum(1 for t, g in zip(truth, guess) if t == g)
tp = counts.get(("at_risk", "at_risk"), 0)
fp = counts.get(("healthy", "at_risk"), 0)
fn = counts.get(("at_risk", "healthy"), 0)
precision = tp / (tp + fp) if tp + fp else 0.0
recall = tp / (tp + fn) if tp + fn else 0.0
f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0

print()
print("accuracy  : %d/%d = %.3f" % (correct, total, correct / total))
print("precision : %d/(%d+%d) = %.3f" % (tp, tp, fp, precision))
print("recall    : %d/(%d+%d) = %.3f" % (tp, tp, fn, recall))
```

Python (continued)

```
print("F1        : %.3f" % f1)
print()
print("per test row")
for row, t, g in zip(test, truth, guess):
    print("  spots %d humidity %2d age %2d -> predicted %-8s (true %-8s) %s"
          % (row[0], row[1], row[2], g, t, "ok" if g == t else "WRONG"))
```

output

```
confusion matrix, rows = truth, columns = prediction
             pred healthy pred at_risk
true healthy          2          1
true at_risk          0          1

accuracy  : 3/4 = 0.750
precision : 1/(1+1) = 0.500
recall    : 1/(1+0) = 1.000
F1        : 0.667

per test row
  spots 2 humidity 45 age 10 -> predicted healthy  (true healthy ) ok
  spots 7 humidity 80 age 25 -> predicted at_risk  (true at_risk ) ok
  spots 4 humidity 58 age 18 -> predicted healthy  (true healthy ) ok
  spots 5 humidity 66 age 20 -> predicted at_risk  (true healthy ) WRONG
```

The single wrong row is the honest part of this example. One healthy plant with moderate spots and humidity is called at risk, because the three nearest rows include one at-risk row. Recall is 1.0 while precision is 0.5: the model catches every at-risk plant and raises one false alarm doing it. Whether that is acceptable depends on what a miss costs, which is a question for the person using the model, not for the metric.

Example 2: four-fold cross-validation comparing three values of k.

Python

```
"""k-fold cross-validation by hand on the same twelve rows."""

plants = [
    (2, 45, 10, "healthy"),
    (1, 40,  5, "healthy"),
    (3, 50, 12, "healthy"),
    (7, 80, 25, "at_risk"),
    (8, 85, 20, "at_risk"),
    (6, 75, 30, "at_risk"),
    (4, 58, 18, "healthy"),
    (5, 68, 22, "at_risk"),
    (9, 90, 18, "at_risk"),
    (5, 66, 20, "healthy"),
    (3, 48,  9, "healthy"),
    (7, 82, 28, "at_risk"),
]

def squared_distance(a, b):
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2 + (a[2] - b[2]) ** 2

def predict(row, train, k):
    ranked = sorted(train, key=lambda r: squared_distance(r, row))
    votes = {}
    for neighbour in ranked[:k]:
        votes[neighbour[3]] = votes.get(neighbour[3], 0) + 1
    return max(votes, key=votes.get)

def fold_accuracy(k, folds=4):
    scores = []
    for fold in range(folds):
        train = [row for i, row in enumerate(plants) if i % folds != fold]
        test = [row for i, row in enumerate(plants) if i % folds == fold]
        correct = sum(1 for row in test if predict(row, train, k) == row[3])
        scores.append(correct / len(test))
    return scores

print("4-fold cross-validation, k-NN with k = 1, 3, 5")
print("%-6s %-34s %s" % ("k", "fold accuracies", "mean"))
for k in (1, 3, 5):
    scores = fold_accuracy(k)
    print("%-6d %-34s %.3f" % (k, " ".join("%.2f" % s for s in scores),
                               sum(scores) / len(scores)))
print()
print("folds are formed by row index, so every fourth row is held out in turn")
```

output

```
4-fold cross-validation, k-NN with k = 1, 3, 5
k      fold accuracies                    mean
1      1.00 0.67 1.00 0.67                0.833
3      1.00 0.67 1.00 0.67                0.833
5      1.00 0.67 1.00 0.67                0.833

folds are formed by row index, so every fourth row is held out in turn
```

All three values of k give the same mean here, which is itself a result: with twelve rows, the differences between k values are within the noise of one or two rows. The right reading is not "k does not matter" but "this data set is too small to choose k", which is a statement about the data and not about k-NN.

Example 3: five random splits, and what a seed buys.

Python

```python
"""The same data, different random splits, and why the seed matters."""

import random
import sys

plants = [
    (2, 45, 10, "healthy"),
    (1, 40,  5, "healthy"),
    (3, 50, 12, "healthy"),
    (7, 80, 25, "at_risk"),
    (8, 85, 20, "at_risk"),
    (6, 75, 30, "at_risk"),
    (4, 58, 18, "healthy"),
    (5, 68, 22, "at_risk"),
    (9, 90, 18, "at_risk"),
    (5, 66, 20, "healthy"),
    (3, 48,  9, "healthy"),
    (7, 82, 28, "at_risk"),
]

def split(seed):
    rng = random.Random(seed)
    order = list(range(len(plants)))
    rng.shuffle(order)
    return ([plants[i] for i in order[:8]],
            [plants[i] for i in order[8:]],
            order[:8], order[8:])

def squared_distance(a, b):
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2 + (a[2] - b[2]) ** 2

def accuracy(train, test, k=3):
    correct = 0
    for row in test:
        ranked = sorted(train, key=lambda r: squared_distance(r, row))
        votes = {}
        for neighbour in ranked[:k]:
            votes[neighbour[3]] = votes.get(neighbour[3], 0) + 1
        if max(votes, key=votes.get) == row[3]:
            correct += 1
    return correct / len(test)

print("five different random splits of the same data")
for seed in (1, 2, 3, 4, 5):
    train, test, train_idx, test_idx = split(seed)
    print("  seed %d: train %s, test %s, k-NN accuracy %.3f"
          % (seed, train_idx, test_idx, accuracy(train, test)))

print()
print("one seed, repeated twice")
for run in (1, 2):
    train, test, _t, _s = split(7)
    print("  run %d: train rows %s, accuracy %.4f" % (run, _t,
                                                      accuracy(train, test)))
print()
print("python %s" % sys.version.split()[0])
```

Python (continued)

```python
print("data checksum (sum of the numbers in the table): %d"
      % sum(row[0] + row[1] + row[2] for row in plants))
```

output

```
five different random splits of the same data
  seed 1: train [7, 11, 0, 8, 5, 6, 3, 10], test [4, 1, 9, 2], k-NN accuracy 0.750
  seed 2: train [9, 11, 3, 4, 7, 6, 8, 2], test [5, 10, 1, 0], k-NN accuracy 1.000
  seed 3: train [1, 7, 10, 0, 6, 11, 4, 5], test [2, 8, 9, 3], k-NN accuracy 0.750
  seed 4: train [11, 2, 8, 10, 5, 0, 9, 7], test [6, 1, 4, 3], k-NN accuracy 1.000
  seed 5: train [10, 2, 11, 7, 1, 3, 6, 0], test [8, 5, 4, 9], k-NN accuracy 0.750

one seed, repeated twice
  run 1: train rows [7, 11, 3, 10, 8, 4, 9, 1], accuracy 1.0000
  run 2: train rows [7, 11, 3, 10, 8, 4, 9, 1], accuracy 1.0000

python 3.13.15
data checksum (sum of the numbers in the table): 1064
```

The same data and the same model score between 0.75 and 1.0 depending only on which rows land in the test set. A single split therefore reports one number from a range; publishing that number without the seed is publishing a number nobody can reproduce. The two runs with seed 7 agree to four decimal places, and the checksum confirms the table was not edited between runs.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reporting accuracy alone	a small class disappears inside it	report precision and recall beside it
Tuning on the test rows	the reported score is optimism	tune on validation rows, test once
Choosing k from one split	the choice is noise on small data	compare across folds or several splits
Forgetting the seed	nobody, including you, can repeat the run	print the seed next to the result
Leaving the mistakes unread	the interesting failure is inside the confusion matrix	print every wrong row with its features
Comparing two models on different splits	the comparison is meaningless	use the same splits for both models

- Say out loud what a miss costs and what a false alarm costs before picking a threshold; the metric cannot choose for you.
- A result you cannot reproduce is a rumour.

11.5 Practice exercises

1. From the confusion matrix in Example 1, compute the false positive rate among healthy rows.
2. A model on a different data set has recall 0.2 and precision 0.95. Describe in one sentence what it does.

3. Why does cross-validation help when only twelve rows exist?
4. In Example 3, which seed would you publish, and why would publishing all five be better?
5. List the five items to record so that a colleague can repeat your run exactly.

Solutions

1. One healthy row out of three was called at risk, so the false positive rate is `1 / 3 = 0.333`.
2. It almost never shouts, and when it does it is nearly right: it misses most of the positive cases while keeping its alarms clean.
3. Because every row is used for testing exactly once, so the estimate uses all the information available and is not tied to one arbitrary subset.
4. All five, with the seed, the split and the mean. Publishing one seed invites the reader to think the score is a property of the model rather than of the split, and the honest summary is the range plus the average.
5. The data and its size, the split rule, the seed, the parameters such as k, and the metrics together with the list of mistaken rows.

11.6 Mini test

1. Which cell counts rows that are at risk and predicted at risk? A) True negative B) True positive C) False positive D) False negative
2. Precision is 0.5 in Example 1. What does that mean? A) Half of all rows are at risk B) Half of the at-risk predictions were on healthy rows C) Half of the at-risk rows were missed D) Half of the features were used
3. What does four-fold cross-validation do with each row? A) Deletes it B) Uses it once in a test fold and three times in training C) Duplicates it D) Uses it only for training
4. True or false: a model with 100 percent accuracy on the test set is always the best model.
5. Why are five splits printed instead of one? A) To make the output longer B) To show how much the score depends on the split C) To avoid using a seed D) To test more data
6. What does the data checksum in Example 3 demonstrate? A) That the model is accurate B) That the input table was not changed between runs C) That the seed is random D) That the sort is stable

Answer key and explanations

1. B - the row and the prediction are both positive, which is a true positive.
2. B - precision is true positives over all positive predictions, so half of the alarms were wrong.
3. B - each row sits in one held-out fold and in the other three training folds.
4. False - a small test set makes any score unstable, and a single number says nothing about which errors were made.
5. B - the spread between 0.75 and 1.0 shows the score is partly a property of the split.
6. B - the checksum is a cheap guard that the data did not change under the experiment.

Chapter 12 - Ethics and the society AI works in

12.1 Summary

- A model learns the regularities of its data, including the unfair ones, and repeats them at scale.
- A rule that never reads a protected attribute can still produce unequal outcomes, because the gap lives in the data rather than in the feature list.
- Fairness has several metrics, they can disagree with each other, and choosing one is a value decision rather than a technical one.
- Demographic parity compares selection rates between groups; equal opportunity compares recall among the qualified members of each group.
- A filter bubble grows when a system optimises for clicks: the offered variety shrinks round after round.
- Anonymity is weaker than it looks, because removing names leaves quasi-identifiers that re-identify people on their own.
- Collecting less data, and stating a system's limits plainly, reduce harm more reliably than any promise.
- Regulation has arrived: the EU AI Act sets out a risk-based framework, and UNESCO has published a global recommendation on the ethics of AI.
- Teams that audit their own systems find problems before the people affected by them do.
- Accountability means a named person can answer for a decision, whatever produced it.

12.2 Concepts

Fairness metrics, and what each one says

Metric	Formula	Value in the example	Reading
Selection rate	selected / group size	A 0.40, B 0.20	twice as many in group A are chosen
Demographic parity difference	rate A - rate B	0.20	the simplest gap measure
Disparate impact ratio	rate B / rate A	0.50	below 0.8 is the usual rule of thumb
Recall on qualified	selected and qualified / qualified	A 0.67, B 0.40	qualified people in B are missed more often
Equal opportunity difference	recall A - recall B	0.27	the gap in who gets a chance

Note the tension: a rule can satisfy demographic parity and still fail equal opportunity, because the two questions are different. Choosing between them is a decision about what kind of equality matters in that setting.

Why removing the attribute is not enough

The example's rule never reads the group column, yet the selection rates differ. The reason is that the inputs correlate with the attribute through the world the data came from. Fairness work therefore has to be measured on outcomes, not on the feature list.

The feedback loop that shrinks variety

output

```
items offered  ----->  what the user opens  ----->  what the system
     ^                                                    | learns to
     |                                                    | offer more
     +----------------------------------------------------+

each round amplifies the last, and the offered set narrows
without anyone deciding to narrow it
```

Anonymity and quasi-identifiers

Idea	Meaning
Direct identifier	a name, a phone number
Quasi-identifier	age band, postcode, gender, which identify jointly
k-anonymity	every combination of quasi-identifiers appears at least k times
Sensitivity	how much damage the leaked value does

The regulatory landscape

Instrument	Character
EU AI Act, regulation 2024/1689	risk-based duties: unacceptable uses banned, high-risk systems need assessment and documentation
UNESCO recommendation on the ethics of AI	a global policy framework agreed by member states
National privacy rules such as the GDPR	limits collection, requires a purpose and a lawful basis

The direction of travel is the same everywhere: document what the system does, measure who it affects, and keep a human answerable for the outcome.

12.3 Examples

Example 1: a fairness audit of a simple hiring rule on a hand-made data set.

Python

```
"""A fairness audit of a simple hiring rule on a hand-made data set."""

applicants = [
    ("A", 7, 93), ("A", 5, 84), ("A", 4, 79), ("A", 2, 66),
    ("A", 6, 88), ("A", 1, 52), ("A", 8, 95), ("A", 3, 74),
    ("A", 5, 81), ("A", 2, 70),
    ("B", 6, 77), ("B", 4, 69), ("B", 8, 82), ("B", 3, 73),
    ("B", 5, 79), ("B", 1, 58), ("B", 7, 84), ("B", 2, 61),
    ("B", 4, 76), ("B", 3, 72),
]

def qualified(exp, score):
    return exp >= 3 and score >= 75

def hired(exp, score):
    return score >= 82        # the rule never sees the group column

print("%-3s %-6s %-6s %-12s %-8s" % ("grp", "exp", "score", "qualified", "hired"))
for group, exp, score in applicants:
    print("%-3s %-6d %-6d %-12s %-8s"
          % (group, exp, score, qualified(exp, score), hired(exp, score)))

def rate(group, subset_filter):
    rows = [row for row in applicants if row[0] == group]
    chosen = [row for row in rows if subset_filter(row)]
    return len(chosen), len(rows)

for group in ("A", "B"):
    chosen, total = rate(group, lambda r: hired(r[1], r[2]))
    print()
    print("group %s: %d of %d selected, selection rate %.2f"
          % (group, chosen, total, chosen / total))

sel_a = rate("A", lambda r: hired(r[1], r[2]))
sel_b = rate("B", lambda r: hired(r[1], r[2]))
rate_a, rate_b = sel_a[0] / sel_a[1], sel_b[0] / sel_b[1]
print()
print("demographic parity difference: %.2f" % (rate_a - rate_b))
print("disparate impact ratio       : %.2f" % (rate_b / rate_a))

rec_a = rate("A", lambda r: qualified(r[1], r[2]) and hired(r[1], r[2]))
rec_b = rate("B", lambda r: qualified(r[1], r[2]) and hired(r[1], r[2]))
recall_a = rec_a[0] / len([r for r in applicants
                           if r[0] == "A" and qualified(r[1], r[2])])
recall_b = rec_b[0] / len([r for r in applicants
                           if r[0] == "B" and qualified(r[1], r[2])])
print("recall on qualified applicants: A %.2f, B %.2f" % (recall_a, recall_b))
print("equal opportunity difference   : %.2f" % (recall_a - recall_b))
print()
print("the rule never reads the group column, yet the gap exists")
```

output

```
grp exp    score  qualified    hired
A   7      93     True         True
A   5      84     True         True
A   4      79     True         False
A   2      66     False        False
A   6      88     True         True
A   1      52     False        False
A   8      95     True         True
A   3      74     False        False
A   5      81     True         False
A   2      70     False        False
B   6      77     True         False
B   4      69     False        False
B   8      82     True         True
B   3      73     False        False
B   5      79     True         False
B   1      58     False        False
B   7      84     True         True
B   2      61     False        False
B   4      76     True         False
B   3      72     False        False

group A: 4 of 10 selected, selection rate 0.40

group B: 2 of 10 selected, selection rate 0.20

demographic parity difference: 0.20
disparate impact ratio       : 0.50
recall on qualified applicants: A 0.67, B 0.40
equal opportunity difference   : 0.27

the rule never reads the group column, yet the gap exists
```

Group A is selected at twice the rate of group B, and qualified people in group B are chosen two times out of five against four times out of six in group A. The rule reads only the test score, so the gap comes from how scores are distributed between the groups, which is a fact about the world that produced the data. A useful audit always prints the group breakdown, because an overall accuracy hides it.

Example 2: a recommender that optimises clicks and loses variety.

Python

```
"""A recommender that only optimises clicks slowly loses variety."""

import random

catalog = (
    [("news-%d" % i, "politics") for i in range(1, 5)]
    + [("sport-%d" % i, "sports") for i in range(1, 5)]
    + [("science-%d" % i, "science") for i in range(1, 5)]
    + [("food-%d" % i, "food") for i in range(1, 5)]
    + [("travel-%d" % i, "travel") for i in range(1, 5)]
)

random.seed(42)
prefs = {"politics": 1.0, "sports": 0.2, "science": 0.2,
         "food": 0.2, "travel": 0.2}

print("the user only ever opens politics items")
print()
for round_no in range(1, 6):
    ranked = sorted(
        ((item, cat, prefs[cat] + random.uniform(-0.05, 0.05))
         for item, cat in catalog),
        key=lambda row: row[2], reverse=True)
    top5 = ranked[:5]
    categories = [cat for _item, cat, _score in top5]
    print("round %d: %s" % (round_no, ", ".join(categories)))
    print("  distinct categories offered: %d of 5" % len(set(categories)))
    clicked = top5[0][1]
    prefs[clicked] *= 2.0            # the system learns what was opened
    for cat in prefs:                # and everything else fades
        if cat != clicked:
            prefs[cat] *= 0.9
    print("  category scores now: %s"
          % ", ".join("%s %.2f" % (c, prefs[c]) for c in sorted(prefs)))
```

output

```
the user only ever opens politics items

round 1: politics, politics, politics, politics, sports
  distinct categories offered: 2 of 5
  category scores now: food 0.18, politics 2.00, science 0.18, sports 0.18, travel 0.18
round 2: politics, politics, politics, politics, food
  distinct categories offered: 2 of 5
  category scores now: food 0.16, politics 4.00, science 0.16, sports 0.16, travel 0.16
round 3: politics, politics, politics, politics, food
  distinct categories offered: 2 of 5
  category scores now: food 0.15, politics 8.00, science 0.15, sports 0.15, travel 0.15
round 4: politics, politics, politics, politics, science
  distinct categories offered: 2 of 5
  category scores now: food 0.13, politics 16.00, science 0.13, sports 0.13, travel 0.13
round 5: politics, politics, politics, politics, sports
  distinct categories offered: 2 of 5
  category scores now: food 0.12, politics 32.00, science 0.12, sports 0.12, travel 0.12
```

No one decided to narrow the catalogue. The system doubled the score of whatever was opened and faded everything else, and after five rounds the politics score is 32 against 0.12 for every other category. The user's own behaviour drove it, which is why the fix cannot be only "recommend better": it has to include a measure of variety that the system is not allowed to destroy.

Example 3: a check for anonymity on a small released table.

Python

```python
"""Removing names is not the same as removing identity: a k-anonymity check."""

records = [
    ("20-29", "00100", "F", "flu"),
    ("20-29", "00100", "F", "flu"),
    ("20-29", "00100", "F", "allergy"),
    ("20-29", "00100", "M", "flu"),
    ("20-29", "00100", "M", "flu"),
    ("20-29", "00100", "M", "flu"),
    ("30-39", "00100", "F", "flu"),
    ("30-39", "00100", "F", "flu"),
    ("30-39", "00100", "F", "flu"),
    ("30-39", "00200", "M", "flu"),
    ("30-39", "00200", "M", "flu"),
    ("40-49", "00300", "F", "depression"),
]

K = 3
groups = {}
for age, zip_code, gender, condition in records:
    groups.setdefault((age, zip_code, gender), []).append(condition)

print("quasi-identifier groups (age band, postcode, gender)")
print("release rule: every group must hold at least %d records" % K)
print()
risky = 0
for key in sorted(groups):
    values = groups[key]
    if len(values) < K:
        risky += 1
    print("  %-30s %d record(s) %-26s %s"
          % (" / ".join(key), len(values), ", ".join(values),
             "ok" if len(values) >= K else "TOO SMALL"))
print()
print("%d of %d groups are too small" % (risky, len(groups)))
print()
print("the single ('40-49', '00300', 'F') row reveals its own value:")
print("anyone who knows a woman in her forties in postcode 00300")
print("can read her condition straight off the table")
```

output

```
quasi-identifier groups (age band, postcode, gender)
release rule: every group must hold at least 3 records

  20-29 / 00100 / F              3 record(s) flu, flu, allergy          ok
  20-29 / 00100 / M              3 record(s) flu, flu, flu              ok
  30-39 / 00100 / F              3 record(s) flu, flu, flu              ok
  30-39 / 00200 / M              2 record(s) flu, flu                   TOO SMALL
  40-49 / 00300 / F              1 record(s) depression                 TOO SMALL

2 of 5 groups are too small

the single ('40-49', '00300', 'F') row reveals its own value:
anyone who knows a woman in her forties in postcode 00300
can read her condition straight off the table
```

Three of the five groups pass, but the two small ones fail the release rule, and one of them holds a single record. De-identification is a property of the released table as a whole, not of

the columns that were removed: a group of one is a person.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating an average score as enough	group gaps disappear inside the mean	report metrics per group
Removing the protected column and stopping	outcomes stay unequal, because the data does	measure outcomes, not the feature list
Choosing a fairness metric silently	the decision is a value judgement	state which metric was chosen and why
Optimising a single engagement number	variety and trust erode over rounds	measure diversity next to engagement
Publishing a table with unique combinations	rows can be re-identified	check group sizes and generalise rare values
Deferring ethics to a review at the end	redesign at the end is the expensive one	bring the audit into the evaluation step

- Write down, before the model is built, who could be harmed by a wrong answer and how they would learn about it.
- If a metric cannot be computed for a group, that group is probably underrepresented in the data, which is itself the finding.

12.5 Practice exercises

1. In Example 1, compute the false positive rate in each group: how many unqualified applicants were hired.
2. The disparate impact ratio is 0.50. Explain what that number means for group B, and state the usual rule of thumb.
3. A team satisfies demographic parity but violates equal opportunity. Describe in one sentence what their selection looks like.
4. Give two changes to the recommender in Example 2 that would preserve variety without removing the learning.
5. A released table keeps a single row per postcode and age band. Neither column alone identifies anyone. Explain why the table can still fail anonymity, and name the fix.

Solutions

1. Group A hired 4 of 10, of which 4 were qualified, so no unqualified applicant was hired and the false positive rate is 0. Group B hired 2, both qualified, so its false positive rate is also 0. The difference between the groups is entirely in who is missed.
2. Group B is selected at half the rate of group A, so qualified people in group B have noticeably less chance; the usual rule of thumb is a ratio below 0.8 as a warning sign,

sometimes called the four-fifths rule.

3. They select at the same rate from both groups, but the people they select in group B are less often the qualified ones, so access looks equal while opportunity does not.
4. Reserve part of the offered list for categories the user has not opened recently, and report the number of distinct categories offered as a metric with a floor; both keep the learning while protecting the range.
5. Because the combination of postcode and age band can be unique, and unique combinations identify people when they are matched against other data. The fix is to generalise the columns, for example wider age bands or larger regions, until every combination appears at least k times.

12.6 Mini test

1. What does demographic parity compare? A) Accuracy between groups B) Selection rates between groups C) Error sizes between features D) The number of features per group
2. Why did the hiring rule produce a gap even though it never read the group column? A) The column was hidden but still used B) The test scores themselves differ between the groups in the data C) The rule was biased by design D) The applicants were chosen at random
3. What is equal opportunity measured on? A) All applicants B) The qualified applicants of each group C) The hired applicants of each group D) The unqualified applicants
4. True or false: a table with names removed but unique combinations of age, postcode and gender can still identify people.
5. What happened to the variety of items in Example 2 after five rounds? A) It increased steadily B) It stayed at five categories C) It settled at two categories out of five D) It became random
6. Which is the best first response when a group metric cannot be computed? A) Publish the result anyway B) Investigate why the group is missing from the data C) Remove the metric D) Average the groups

Answer key and explanations

1. B - demographic parity compares the share of each group that is selected, and the difference between those shares is the measure.
2. B - the gap comes from the data, so hiding the attribute does not close it; only measurement and intervention do.
3. B - equal opportunity compares recall among the qualified members of each group, which is why it answers a different question from demographic parity.
4. True - quasi-identifiers can be unique, and unique combinations re-identify people even without names.
5. C - the top five offered only two of the five categories by the end, and the scores show why.
6. B - a missing group is a coverage problem, and coverage problems are findings rather than excuses.

Chapter 13 - Where to go next

13.1 Summary

- The methods in this volume are foundations: search, probability, nearest neighbours, linear models, perceptrons and honest evaluation.
- A plan beats motivation: a fixed weekly slot with a fixed project is what finishes a course.
- A twelve-week schedule covers the foundations first and then one chosen track at a time.
- A capstone with a written report and an honest error analysis is the most convincing evidence of skill.
- A self-assessment turns "I think I know this" into a list of gaps that can be attacked.
- A notebook of the mistakes you already found makes each new topic cheaper.
- Reading one paper a week, slowly, becomes a skill that pays off later rather than immediately.
- Free datasets and open communities keep practice going once the course is finished.
- Explaining a topic out loud is the fastest test of whether you actually understand it.
- The next volumes of this series continue where this one stops, with data handling and model building in more depth.

13.2 Concepts

A shape for the next twelve weeks

Block	Focus	Output
Weeks 1-4	Python, probability, search, regression	five scripts and one written explanation each
Weeks 5-8	one chosen track, in blocks	a small project that runs end to end
Weeks 9-10	a second, shorter block	a comparison of two methods on the same data
Week 11	capstone	one model, one report, one error analysis
Week 12	review	retake every mini test and list what is still weak

Choosing what to study next

If you enjoyed	Go next	Because
Search and minimax	planning, constraint solving, game playing	the same ideas with larger spaces
Bayes and Naive Bayes	probabilistic graphical models, text classification	the models grow but the law does not

If you enjoyed	Go next	Because
k-NN and regression	trees, ensembles, gradient boosting	the same questions, more expressive answers
Neural networks	backpropagation variants, convolutional and recurrent models	deeper networks, same training loop
Fairness and audits	responsible AI practice, privacy engineering	measurement work in real organisations
Everything	a data structures and algorithms volume	every method above rests on them

Habits that keep the skill

Habit	Payoff
Write the expected output before running	you notice when the program is wrong
Keep the seed next to every number	your results stay reproducible
Retake a mini test after a week	spaced recall beats a single reading
Explain a method without notes	gaps appear instantly
Read one paper a week, twice	the second reading is where the value is

13.3 Examples

Example 1: a twelve-week plan generator, built from the tracks you pick.

Python

```
"""A twelve-week study plan built from the tracks you want to follow."""

FOUNDATION = [
    ("Python for data", "write five scripts that read a list and print statistics"),
    ("Probability and odds", "redo three Bayes problems by hand, then in code"),
    ("Search and games", "breadth-first search on a map, minimax on tic-tac-toe"),
    ("Neighbours and lines", "k-NN and linear regression written from scratch"),
]

TRACKS = {
    "machine learning": ("Supervised and unsupervised practice",
                         "cross-validate a k-NN model you wrote yourself"),
    "deep learning": ("Neural networks from scratch",
                      "train a two-layer network on XOR, then on digits"),
    "nlp": ("Text and language",
            "bag-of-words plus Naive Bayes on your own mailbox"),
    "computer vision": ("Images as numbers",
                        "find edges with a filter you write by hand"),
    "ethics": ("Fairness and audit",
               "measure the group gaps in a data set you build"),
}

chosen = ["maths", "machine learning", "deep learning", "nlp", "ethics"]
chosen = [t for t in chosen if t in TRACKS]

rows = []
for focus, practice in FOUNDATION:
    rows.append((focus, practice))

week = len(rows) + 1
seen = {}
while week <= 10:
    name = chosen[(week - 5) % len(chosen)]
    seen[name] = seen.get(name, 0) + 1
    title, practice = TRACKS[name]
    rows.append(("%s, block %d" % (title, seen[name]), practice))
    week += 1

rows.append(("Capstone project",
             "one model, one written report, one honest error analysis"))
rows.append(("Review and next steps",
             "retake every mini test, then pick a deeper volume"))

print("tracks chosen: %s" % ", ".join(chosen))
print()
for number, (focus, practice) in enumerate(rows, 1):
    print("week %-3d %s" % (number, focus))
    print("        %s" % practice)

hours = len(rows) * 5 * 2
print()
print("%d hours at two hours a day, five days a week" % hours)
print("rule of thumb: one third reading, one third coding, one third testing")
```

output

```
tracks chosen: machine learning, deep learning, nlp, ethics

week 1   Python for data
         write five scripts that read a list and print statistics
week 2   Probability and odds
         redo three Bayes problems by hand, then in code
week 3   Search and games
         breadth-first search on a map, minimax on tic-tac-toe
week 4   Neighbours and lines
         k-NN and linear regression written from scratch
week 5   Supervised and unsupervised practice, block 1
         cross-validate a k-NN model you wrote yourself
week 6   Neural networks from scratch, block 1
         train a two-layer network on XOR, then on digits
week 7   Text and language, block 1
         bag-of-words plus Naive Bayes on your own mailbox
week 8   Fairness and audit, block 1
         measure the group gaps in a data set you build
week 9   Supervised and unsupervised practice, block 2
         cross-validate a k-NN model you wrote yourself
week 10  Neural networks from scratch, block 2
         train a two-layer network on XOR, then on digits
week 11  Capstone project
         one model, one written report, one honest error analysis
week 12  Review and next steps
         retake every mini test, then pick a deeper volume

120 hours at two hours a day, five days a week
rule of thumb: one third reading, one third coding, one third testing
```

The chosen tracks fill the middle weeks, the foundation always comes first, and the last two weeks are reserved for a project and a review. Change the list of chosen tracks and print it again; the schedule moves with the list, which is the whole point of writing it as a program.

Example 2: a self-assessment that lists what to study next.

Python

```
"""A self-assessment that points at what to study next."""

checks = [
    ("Python basics", "I can write a loop, a function and a dict without help", True),
    ("Python basics", "I can read a file and split a line into fields", True),
    ("Maths for AI", "I know what a probability is and can multiply two of them", True),
    ("Maths for AI", "I can read a graph of a straight line", False),
    ("Problem solving", "I can explain breadth-first search in my own words", True),
    ("Problem solving", "I can hand-trace minimax on a four-leaf tree", False),
    ("Machine learning", "I have trained a model on data I collected myself", False),
    ("Machine learning", "I know what a train or test split protects me from", True),
    ("Neural networks", "I have written a forward pass in code", False),
    ("Neural networks", "I know why a single neuron cannot learn XOR", True),
    ("Ethics", "I can name two ways a model can be unfair", True),
    ("Ethics", "I have checked a data set for group gaps", False),
]

scores = {}
totals = {}
for area, statement, answered in checks:
    totals[area] = totals.get(area, 0) + 1
    scores[area] = scores.get(area, 0) + (1 if answered else 0)

print("%-18s %-8s %s" % ("area", "score", "readiness"))
print("-" * 58)
for area in sorted(totals):
    score, total = scores[area], totals[area]
    bar = "#" * score + "." * (total - score)
    verdict = ("ready" if score == total else
               "almost there" if score * 2 >= total else "study this")
    print("%-18s %d/%d %-6s %s" % (area, score, total, bar, verdict))

print()
weak = [a for a in sorted(totals) if scores[a] * 2 < totals[a]]
if weak:
    print("next up: %s" % ", ".join(weak))
    print("start with the concepts section, then retake the mini test")
else:
    print("all areas at least half covered: go build something")
```

output

```
area               score    readiness
----------------------------------------------------------
Ethics             1/2 #.     almost there
Machine learning   1/2 #.     almost there
Maths for AI       1/2 #.     almost there
Neural networks    1/2 #.     almost there
Problem solving    1/2 #.     almost there
Python basics      2/2 ##     ready

all areas at least half covered: go build something
```

The example answers are fixed so the output is reproducible; answer them for yourself and the list of weak areas becomes a to-do list. Note that the bar is honest: every area except Python is half covered, and the program says so rather than encouraging you to move on.

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Collecting courses without projects	certificates do not transfer skill	build something that runs and that you can explain
Studying tracks in the order they appear	weaker foundations make later chapters slower	fix the basics first, then branch
Skipping the review week	the month's details fade	retake the mini tests before starting something new
Choosing a capstone with no data	most of the work becomes data hunting	pick a problem where you already have the data
Never writing about what you built	nobody, including you, can see the reasoning	one page per project: data, method, errors, next step
Reading a paper once	the second reading is the one that pays	read twice, then summarise in three sentences

- Keep a file of the bugs you actually hit; it is the most personal and the most useful reference you will own.
- Finish the plan before starting a new one, even if a second plan looks more interesting.

13.5 Practice exercises

1. Write a four-week plan for yourself with one measurable outcome per week.
2. Choose one track from the table of concepts and list the three things you would need to learn to start it.
3. Why is a written error analysis worth more than a high score on a test?
4. Design a capstone project around data you already have access to, and name the metric you would report.
5. Explain, without notes and in under a minute, why one perceptron cannot learn XOR.

Solutions

1. Any plan with one measurable outcome per week is acceptable, for example: week 1, five scripts that read a file; week 2, three Bayes problems solved by hand and checked in code; week 3, breadth-first search and minimax running on a data set you chose; week 4, a k-NN model with a split, a baseline and a confusion matrix.
2. Answers vary. For neural networks, for instance: the chain rule in more than one dimension, the behaviour of the sigmoid and its derivative, and how a training loop updates every weight; each has a concrete practice task.
3. Because the analysis shows what the model does not know, which is the part that decides whether a deployment is safe or useful; a score hides that.
4. For example, a model predicting whether a message in your own mailbox is important, using the spam data set idea as a template, with precision and recall reported per class.
5. Because the four rows of the XOR truth table put the two classes on opposite corners, and one straight boundary cannot separate opposite corners.

13.6 Mini test

1. Which comes first in the twelve-week plan? A) The capstone B) The foundation blocks C) The fairness track D) The review
2. What is the purpose of the self-assessment in Example 2? A) To give a grade B) To turn vague confidence into a list of gaps C) To replace the mini tests D) To measure hours studied
3. True or false: a capstone project is optional because the mini tests already prove the skill.
4. What does the plan print when a chosen track is unknown? A) A crash B) The unknown track is filtered out before the schedule is built C) A blank week D) The foundation is skipped
5. Which habit is recommended for reading research papers? A) Read once, quickly, many papers B) Read twice, then summarise in three sentences C) Skip the introduction D) Read only the conclusion
6. What is the most convincing evidence of skill, according to this chapter? A) A list of finished courses B) A project that runs, with a written report and an error analysis C) A collection of code snippets D) A high score on one test

Answer key and explanations

1. B - the four foundation weeks come first, and the chosen tracks fill the middle weeks.
2. B - the bars make the gaps visible, which is what a plan needs to start from.
3. False - a project exercises data collection, splitting, fitting, measuring and writing, which no test covers.
4. B - the list is filtered against the known tracks, so an unknown name never reaches the schedule.
5. B - the second reading is where the value is, and the summary is the test of understanding.
6. B - a working project with an honest error analysis shows the whole pipeline, not one metric.

Cheat sheet

14.1 Definitions

Term	One line
Artificial intelligence	software for tasks we would call intelligent if a person did them
Rule system	behaviour written as `if` statements by a person
Machine learning	rules derived from examples instead of from a person
State	a complete description of a situation at one moment

Term	One line
State space	every state reachable from the start
Game tree	positions as nodes, moves as children, values at the leaves
Prior	belief before the evidence
Likelihood	how expected the evidence is under one hypothesis
Posterior	belief after the evidence
Label	the answer a supervised model predicts
Baseline	a trivial rule a real model must beat
Confusion matrix	truth against prediction, in counts

14.2 Search

Method	Order	Guarantees	Memory
Breadth-first	level by level	fewest actions	a whole level
Depth-first	one branch deep	only that it exists	one branch
Iterative deepening	limits 0, 1, 2, ...	fewest actions	one branch
Uniform cost	cheapest distance first	cheapest distance	all open states

Action	Code shape
remember where a state came from	`parent[child] = state`
avoid repeats	`seen = {start}`
breadth-first queue	`deque`, `popleft()`
depth-first stack	`list`, `pop()`
rebuild the path	walk the parents back to the start

14.3 Games

Idea	Rule
Maximiser	takes the largest child value
Minimiser	takes the smallest child value
Depth-adjusted win score	`10 - empty cells` for a win
Alpha	best the maximiser can guarantee
Beta	best the minimiser can guarantee

Idea	Rule
Prune	when `beta <= alpha`, skip the remaining children
Chance node	the weighted average of the children

14.4 Probability and Bayes

Formula	Use
`P(not A) = 1 - P(A)`	complement
`P(A or B) = P(A) + P(B)`	exclusive outcomes
`P(A and B) = P(A) * P(B)`	independent events
`P(A and B) = P(A) * P(B given A)`	the general rule
`P(E) = sum of P(E given H) * P(H)`	total probability
posterior odds = prior odds x likelihood ratio	the odds form
posterior = `P(E given H) * P(H) / P(E)`	the long form

Trap	Reminder
Base rate	an accurate test on a rare condition is mostly wrong when positive
Conditional flip	label every probability with what is known
Small samples	running frequency needs many trials

14.5 Naive Bayes

Step	Action
1	count messages per class, take logarithms of the priors
2	count words per class, and the class totals
3	vocabulary `V` is the number of distinct training words
4	`P(word given class) = (count + 1) / (total + V)`
5	add one log likelihood per word, compare the two totals
6	larger total names the verdict

14.6 Neighbours and lines

Item	Formula or rule
Euclidean distance	`sqrt((x1-a1)^2 + (x2-a2)^2)`

Item	Formula or rule
Tie-break	weight each vote by `1 / distance`
Min-max scaling	`(v - min) / (max - min)`
Mean squared error	average of `(prediction - observed)^2`
Gradient of `w`	average of `2 * error * x`
Gradient of `b`	average of `2 * error`
Closed-form slope	`covariance(x, y) / variance(x)`
`R^2`	`1 - sum of squared residuals / total sum of squares`

14.7 Neural networks

Part	Meaning
Pre-activation	`z = w1*x1 + w2*x2 + b`
Sigmoid	`1 / (1 + exp(-z))`
Sigmoid derivative	`a * (1 - a)`
Output delta	`(y - target) * y * (1 - y)`
Hidden delta	`output delta * output weight * h * (1 - h)`
Weight gradient	`delta * activation that fed the weight`
Update	`weight = weight - lr * average gradient`

Failure	Cause
Loss flat	learning rate too small, or a flat region
Loss rising	learning rate too large
Predictions confidently wrong	wrong targets, or the wrong function was trained
One perceptron stuck on XOR	the truth table is not separable by a line

14.8 Evaluation

Metric	Formula	Answers
Accuracy	correct / total	overall share right
Precision	tp / (tp + fp)	of the alarms, how many were right
Recall	tp / (tp + fn)	of the positives, how many were found

Metric	Formula	Answers
F1	harmonic mean of precision and recall	one number for both
Selection rate	selected / group	who gets chosen
Demographic parity difference	rate A - rate B	the size of the gap
Disparate impact ratio	rate B / rate A	below 0.8 warns
Equal opportunity difference	recall A - recall B	gap among the qualified

Habit	Why
baseline first	keeps any score honest
metrics per class	hides nothing in the average
inspect the mistakes	the failures are the next task
print the seed	makes the run repeatable

14.9 Ethics checklist

Check	Question
Group metrics	does every group have a score, or is one missing
Fairness metric	which one was chosen, and why that one
Outcome gaps	are the differences measured, not just the feature list
Diversity	does the offered set stay wide enough
Quasi-identifiers	does every combination appear at least k times
Accountability	which named person answers for a wrong decision
Honesty	are the system's limits written down where users can read them

14.10 Python quick reference

Need	Tool
deterministic randomness	`random.seed(42)` or `random.Random(seed)`
first-in first-out queue	`collections.deque`
most common label	`max(set(labels), key=labels.count)`
logarithms	`math.log`, `math.exp`

Need	Tool
safe sigmoid	branch on the sign of `z` before calling `exp`
sorting by a key	`sorted(rows, key=lambda row: ...)`
counting	a plain dictionary with `.get(key, 0)`

Final test

1. Which statement best defines artificial intelligence as this volume uses the term? A) Any program that runs on a server B) Software that performs tasks we would call intelligent if a person did them C) A robot shaped like a person D) A database large enough to hold the internet
2. What does the Turing test measure? A) Whether a machine is conscious B) Whether a judge can tell machine text from human text C) Whether a machine can move like a person D) Whether a machine is faster than a person
3. True or false: passing a test built on word count and hedging words proves that a system understands language.
4. Which search returns a route with the fewest actions? A) Depth-first search B) Breadth-first search C) A random walk D) A greedy walk
5. In the water jug program, what does the `seen` set prevent?
6. What value does a minimiser node of a game tree return? A) The largest child value B) The smallest child value C) The average of the child values D) The value of the first child
7. What does alpha-beta pruning change about the answer of a minimax search? A) It makes the answer better B) Nothing; it only skips branches that cannot change it C) It removes the need for leaf values D) It makes the search complete
8. What are the odds of an event with probability 0.5?
9. A test has sensitivity 0.99, specificity 0.95 and a base rate of 0.001. Roughly what share of positive results is correct? A) 99 percent B) 50 percent C) 2 percent D) 95 percent
10. Why does the Naive Bayes program add logarithms instead of multiplying probabilities?
11. What does Laplace smoothing change in the word likelihoods?
12. Which type of machine learning needs no labels? A) Supervised learning B) Unsupervised learning C) Reinforcement learning D) Both supervised and unsupervised
13. What is a baseline?
14. With `k = 1`, what score does k-NN get on its own training rows, and why is that score useless?
15. What does feature scaling change in a k-NN model?
16. In the linear regression example, what are the fitted slope and intercept?
17. Why can a single perceptron not learn XOR?
18. What is multiplied by the learning rate in a weight update?
19. In the XOR training table, what happened at a learning rate of 0.1?

20. What does a confusion matrix count?
21. A model has precision 0.5 and recall 1.0. Describe its behaviour in one sentence.
22. Why must the random seed be recorded with a result?
23. In the hiring example, what does a disparate impact ratio of 0.50 mean?
24. What does a k-anonymity check count, and which result is flagged?
25. Name two items you must record so that someone else can repeat your experiment exactly.

15.1 Final test answers

1. B - the definition is about the kind of task, not about hardware, shape or data size.
2. B - the test compares typed behaviour against a judge's ability to tell the answerers apart, and says nothing about understanding.
3. False - a shallow judge can be fooled by a rule, so passing it proves only that the answer text resembles whatever the judge looks for.
4. B - expanding level by level means no shorter route can be waiting behind a longer one.
5. It stops a state that has already been expanded from being expanded again, which prevents loops and keeps the search finite.
6. B - a minimiser takes the worst of its children, exactly as a maximiser takes the best of its own.
7. B - pruning only removes branches whose value cannot change the answer, so the root value is identical.
8. Odds of 1 to 1, because `0.5 / (1 - 0.5) = 1`.
9. C - with a rare condition the false positives from the healthy group dominate, so only about 2 percent of positive results are correct.
10. Because multiplying many small probabilities underflows to zero, while logarithms turn the products into sums and keep the comparison exact.
11. It adds one to every word count and the vocabulary size to every class total, so a word never seen in a class keeps a small non-zero probability instead of zeroing the whole message.
12. B - unsupervised learning works on unlabelled rows, for example by grouping them.
13. A trivial rule, such as always predicting the most common label, whose score any real model must beat before it counts as interesting.
14. It scores 100 percent because each row is its own nearest neighbour at distance zero, and the score measures memory rather than the ability to generalise, so it says nothing about unseen rows.
15. It decides which rows are close: unscaled, the largest column dominates every distance, while min-max scaling lets every column contribute.
16. `w = 0.931429` and `b = 1.14`, which the closed form and gradient descent agree on.
17. Because the two classes of XOR sit on opposite corners of the input square, and no single straight boundary separates them; a hidden layer is needed.

18. The averaged gradient of the loss with respect to that weight, computed from the backward pass.
19. The loss stalled near 0.124 and only two or three of the four rows were right, so the setup was undertrained rather than the seed unlucky.
20. How many rows have each pair of true label and predicted label.
21. It finds every positive case but raises an alarm on healthy rows too: only half of its alarms are correct.
22. Because the seed fixes the starting weights and any random split, so the printed numbers can be reproduced line by line; without it the result cannot be checked.
23. Group B was selected at half the rate of group A, which is well below the usual 0.8 rule of thumb and indicates a substantial gap in who gets chosen.
24. It counts how many rows share each combination of quasi-identifiers, and flags any group smaller than the chosen `k`, because a small group can be matched against other data to re-identify a person.
25. Any two of: the data and its size, the split rule, the seed, the parameters such as `k`, and the metrics together with the list of mistaken rows.

Sources

Consulted while writing this volume. The topic order follows the free course Elements of AI, published by MinnaLearn and the University of Helsinki; every explanation, example and test here is original writing, and the course pages themselves were used only as a table of contents.

Course pages from the source site:

- Elements of AI, the course home: https://www.elementsofai.com/
- Elements of AI, part one, introduction to AI: https://course.elementsofai.com
- Elements of AI, part two, building AI: https://buildingai.elementsofai.com

Reference documentation used while checking the methods:

- scikit-learn, nearest neighbours: https://scikit-learn.org/stable/modules/neighbors.html
- scikit-learn, Naive Bayes: https://scikit-learn.org/stable/modules/naive_bayes.html
- scikit-learn, linear models: https://scikit-learn.org/stable/modules/linear_model.html
- scikit-learn, preprocessing and scaling: https://scikit-learn.org/stable/modules/preprocessing.html
- scikit-learn, clustering and k-means: https://scikit-learn.org/stable/modules/clustering.html
- scikit-learn, model evaluation metrics: https://scikit-learn.org/stable/modules/model_evaluation.html

Python standard library documentation, for the behaviour the examples rely on:

- `random`, seeded generators and `Random`: https://docs.python.org/3/library/random.html
- `math`, logarithms and exponentials: https://docs.python.org/3/library/math.html

- `collections`, deque and Counter: https://docs.python.org/3/library/collections.html
- `statistics`, mean and variance: https://docs.python.org/3/library/statistics.html

Background on the theory and the philosophy:

- Bayes' theorem: https://en.wikipedia.org/wiki/Bayes%27_theorem
- Base rate fallacy: https://en.wikipedia.org/wiki/Base_rate_fallacy
- k-nearest neighbours algorithm: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- Linear regression: https://en.wikipedia.org/wiki/Linear_regression
- Perceptron: https://en.wikipedia.org/wiki/Perceptron
- Backpropagation: https://en.wikipedia.org/wiki/Backpropagation
- Minimax: https://en.wikipedia.org/wiki/Minimax
- Alpha-beta pruning: https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning
- Q-learning: https://en.wikipedia.org/wiki/Q-learning
- K-means clustering: https://en.wikipedia.org/wiki/K-means_clustering
- Turing test: https://en.wikipedia.org/wiki/Turing_test
- Chinese room: https://en.wikipedia.org/wiki/Chinese_room
- The Chinese Room argument, Stanford Encyclopedia of Philosophy: https://plato.stanford.edu/entries/chinese-room/
- Turing's 1950 paper "Computing Machinery and Intelligence": https://academic.oup.com/mind/article-pdf/LIX/236/433/30123314/lix-236-433.pdf

Ethics, fairness and regulation:

- UNESCO, recommendation on the ethics of artificial intelligence: https://www.unesco.org/en/artificial-intelligence/recommendation-ethics
- EU AI Act, explainer and full text: https://artificialintelligenceact.eu/
- EU AI Act, the official text of regulation 2024/1689: https://eur-lex.europa.eu/eli/reg/2024/1689/oj
- NIST AI Risk Management Framework: https://www.nist.gov/itl/ai-risk-management-framework
- Stanford HAI, the AI Index report: https://hai.stanford.edu/ai-index